

Go Learning Notes (สรุปเนื้อหาบทเรียน Golang)

ยินดีต้อนรับสู่โน้ตสรุปการเรียนรู้ภาษา Go (Golang) ครับ สามารถเลือกดูหัวข้อเนื้อหาแยกตามหมวดหมู่ได้ตามด้านล่างนี้:

🔧 คำสั่ง Go CLI พื้นฐานที่จำเป็นต้องรู้ (Essential Go Commands)

รวมคำสั่งภาษา Go (Command-Line) ที่ใช้งานบ่อยในการจัดการโปรเจกต์ Module และ Dependencies:

1. การจัดการ Module และ Dependencies

- `go mod init <module-name>` : สร้างไฟล์ `go.mod` เพื่อเริ่มต้นจัดการ Go Module ในโปรเจกต์ใหม่ (เช่น `go mod init myapp`)
- `go get <package-path>` : ดาวน์โหลดและติดตั้งแพ็คเกจ/ไลบรารีภายนอกเข้ามาใช้งานในโปรเจกต์ (เช่น `go get github.com/gofiber/fiber/v2`)
- `go mod tidy` : ลบ dependencies ที่ไม่ได้ใช้ออกจาก `go.mod` และเติม dependencies ที่ยังขาดอยู่อัตโนมัติ (ควรใช้เสมอเมื่อเพิ่ม/ลบไลบรารี)
- `go mod download` : ดาวน์โหลดแพ็คเกจทั้งหมดที่ระบุไว้ใน `go.mod` ลงเครื่อง

2. การรันและการคอมไพล์โปรเจกต์ (Build & Run)

- `go run <file.go>` หรือ `go run .` : คอมไพล์และรันโปรเจกต์ทันทีโดยไม่ต้องสร้างไฟล์ Executable การ (เหมาะสำหรับทดสอบ/พัฒนา)
- `go build` : คอมไพล์โปรเจกต์ให้ออกมาเป็นไฟล์ Binary Executable (เช่น `.exe` บน Windows) สำหรับนำไป Deploy
- `go install` : คอมไพล์และติดตั้งไฟล์ Binary ไว้ในโฟลเดอร์ `$GOPATH/bin` เพื่อให้เรียกใช้งานคำสั่งนั้นได้จากทุกที่บนระบบ

3. การทดสอบ จัดฟอร์แมต และตรวจสอบระบบ (Testing & Tools)

- `go test ./...` : รัน Unit Test ทั้งหมดในโปรเจกต์และ sub-packages
- `go fmt ./...` : จัดรูปแบบจัดหน้าโค้ด (Formatting) ทุกไฟล์ในโปรเจกต์ให้เป็นมาตรฐานของ Go โดยอัตโนมัติ
- `go env` : แสดงค่าการตั้งค่า Environment Variables ทั้งหมดของ Go (เช่น `GOPATH`, `GOROOT`)

4. 🔄 ตารางเปรียบเทียบคำสั่ง Go CLI vs npm CLI (Node.js)

สำหรับผู้ที่ยังคุ้นเคยกับ Node.js / npm สามารถเปรียบเทียบคำสั่งที่ทำหน้าที่ใกล้เคียงกันได้ดังนี้:

การทำงาน (Action / Task)	Go CLI (Golang)	npm CLI (Node.js)	คำอธิบายเพิ่มเติม
สร้างโปรเจกต์ใหม่	<code>go mod init <name></code>	<code>npm init / npm init -y</code>	สร้างไฟล์ Manifest (<code>go.mod</code> vs <code>package.json</code>)

การทำงาน (Action / Task)	Go CLI (Golang)	npm CLI (Node.js)	คำอธิบายเพิ่มเติม
ติดตั้ง Library เพิ่มเติม	go get <package>	npm install <package>	ดาวน์โหลดและบันทึกแพ็คเกจลง Manifest File
ติดตั้ง Dependencies ทั้งหมด	go mod download	npm install	ดาวน์โหลดแพ็คเกจตามรายการในไฟล์ Manifest
จัดการ/เคลียร์ Dependencies	go mod tidy	npm prune / npm uninstall	go mod tidy จะช่วยลบตัวไม่ใช่ และเติมตัวที่ขาดให้อัตโนมัติ
สั่งรันโปรเจกต์	go run . / go run main.go	npm start / node index.js	คอมไพล์และสั่งรันโปรเจกต์ในโหมดพัฒนา
คอมไพล์สำหรับ Production	go build	npm run build	สร้างไฟล์คอมไพล์ (Binary Executable vs JS Bundle)
ติดตั้ง CLI Tool แบบ Global	go install <path>	npm install -g <package>	ติดตั้งคำสั่งไว้ในระบบเพื่อเรียกใช้ได้ทุกที่
รัน Unit Test	go test ./...	npm test	รันชุดทดสอบความถูกต้องของโค้ด
จัดฟอร์แมตโค้ดอัตโนมัติ	go fmt ./...	npx prettier --write .	จัดหน้าโค้ดให้เรียบร้อยตามมาตรฐานภาษา
ไฟล์ข้อมูล Dependencies	go.mod	package.json	ไฟล์หลักระบุเวอร์ชันและรายชื่อแพ็คเกจ
ไฟล์ Lock Version & Hash	go.sum	package-lock.json	ไฟล์เก็บ Checksum ยืนยันความถูกต้องของเวอร์ชัน

สารบัญเนื้อหา (Table of Contents)

1. เปรียบเทียบ Array vs Slice (Array & Slice)

สรุปโครงสร้างข้อมูลแบบลำดับใน Go:

- ข้อแตกต่างระหว่าง **Array** และ **Slice**: ตารางเปรียบเทียบขนาด, ชนิดข้อมูล, การส่งค่าเข้าฟังก์ชัน และการเพิ่มข้อมูล
- โครงสร้างภายใน **Slice Header**: ทำความเข้าใจ Pointer, Length (len), และ Capacity (cap)
- ตัวอย่างโค้ดการใช้งาน: การประกาศ, append() , และการตัดแบ่ง Slicing ([low:high])
- การแปลงข้อมูล (**Conversion**): แปลง Array เป็น Slice (arr[:]) และแปลง Slice เป็น Array ([N]T(slice) / Pointer)

2. การใช้งาน Map (Map in Go)

สรุปโครงสร้างข้อมูลแบบ Key-Value Pairs พร้อมความสำคัญและตัวอย่างการประยุกต์ใช้งานจริง:

- **ความสำคัญและการค้นหาข้อมูล $O(1)$:** ประสิทธิภาพการค้นหาข้อมูลระดับ $O(1)$ ด้วย Hash Table และความยืดหยุ่นในการใช้ Key ประเภทต่างๆ
- **คุณสมบัติและข้อควรระวังสำคัญ:** Unordered, Reference Type, Zero Value (`nil` map panic) และเรื่อง **Thread-Safety** (`concurrent map writes`)
- **การจัดการข้อมูลและ Comma-ok Idiom:** การใช้ `make()` , `delete()` , `len()` และการเช็คการมีอยู่ด้วย `val, ok := m[key]`
- **ตัวอย่างการใช้งานในระบบจริง (Real-World Use Cases):** การใช้ Map สร้าง Set ป้องกันค่าซ้ำ (`map[string]bool`), การทำ Frequency Counter (`counts[word]++`), In-Memory Index/Caching และการจัดกลุ่มข้อมูล (Group By)

3. การใช้งาน Struct (Struct in Go)

สรุปการใช้งานโครงสร้างข้อมูลแบบกำหนดเอง (Custom Data Types):

- **การประกาศและสร้าง Struct:** การกำหนดชนิดข้อมูลเฉพาะสำหรับอธิบายวัตถุ
- **Nested Struct:** การสร้าง Struct ซ้อน Struct เพื่อจัดกลุ่มข้อมูลให้เป็นระบบ
- **การใช้งานร่วมกับ Map/Slice:** ตัวอย่างการสร้าง `map[string]Student` และการอ่านค่าด้วย Dot Notation (`student.Address.City`)

4. เปรียบเทียบ Function vs Method (Function vs Method)

สรุปความแตกต่างระหว่างฟังก์ชันทั่วไปและเมธอดในภาษา Go:

- **ตารางเปรียบเทียบข้อแตกต่าง:** การผูกติดกับ Type, การเรียกใช้งาน, การประกาศ Syntax และ Receiver
- **Value Receiver vs Pointer Receiver:** ข้อแตกต่างของการส่งค่าแบบ Pass by Value (`func (s Student)`) และ Pass by Reference (`func (s *Student)`)
- **ตัวอย่างโค้ดและเกณฑ์การใช้งาน:** ตัวอย่างเปรียบเทียบการเรียกใช้งาน Function และ Method พร้อมคำแนะนำเมื่อไหร่ควรเลือกใช้แบบใด

5. การใช้งาน Interface (Interface in Go)

สรุปการใช้งาน Interface และพฤติกรรม Polymorphism ใน Go:

- **Implicit Implementation:** เข้าใจการทำตามสัญญา Interface โดยไม่ต้องใช้คีย์เวิร์ด `implements`
- **Polymorphism:** ตัวอย่างการส่งอินสแตนซ์ต่าง Struct เช่น `Circle` และ `Square` เข้าฟังก์ชันคำนวณพื้นที่ `printArea(s Shape)` รวมถึงตัวอย่าง `Speaker`
- **แนวคิดสำคัญ:** Empty Interface (`interface{}` / `any`), Type Assertion และการลดความยึดติดของโค้ด (Decoupling)

6. การใช้งาน Pointer (Pointer in Go)

สรุปการจัดการตำแหน่งหน่วยความจำและการใช้งาน Pointer ใน Go:

- **สัญลักษณ์สำคัญ:** สัญลักษณ์ `&` (Address-of) และ `*` (Dereference / Pointer Type)
- **หลักการทํางานและ Struct Pointer:** การส่ง Pointer เข้าฟังก์ชันเพื่อแก้ไขข้อมูลต้นทาง (`giveRaise(&emp, 15)`) พร้อมความสามารถ **Automatic Dereferencing** (`e.Salary`)
- **ประโยชน์และการเลือกใช้:** ประโยชน์ของการประหยัดหน่วยความจำ และเมื่อไหร่ที่ควรเลือกใช้ Pointer

7. การจัดการข้อผิดพลาด (Error Handling in Go)

สรุปแนวคิดและการจัดการข้อผิดพลาดในภาษา Go:

- **error Interface & if err != nil**: ทำความเข้าใจการจัดการข้อผิดพลาดแบบ Explicit Error Handling
- **การสร้าง Error**: วิธีการใช้ `errors.New()`, `fmt.Errorf()` และการสร้าง **Custom Error Struct** (`LoginError`)
- **แนวปฏิบัติที่ดี (Best Practices)**: การวางตำแหน่ง Return Error ตัวสุดท้าย, Early Return (Guard Clause) และข้อควรระวังเรื่อง `panic`

8. การใช้งาน HTTP Server & Client (HTTP in Go)

สรุปการสร้าง HTTP Web Server และการส่ง Request ด้วย Package `net/http`:

- **การสร้าง Web Server**: การใช้ `http.HandleFunc()` และ `http.ListenAndServe()` เพื่อให้บริการ API
- **การจัดการ Request & Response**: การอ่าน Query Parameters, Headers, Request Body และการส่ง JSON Response
- **การใช้งาน HTTP Client**: การส่ง GET/POST Request ด้วย `http.Get()`, `http.Post()` และ `http.Client`

9. การใช้งาน Go Fiber Framework (Fiber in Go)

สรุปการพัฒนา Web API ด้วย Fiber Framework (Express-style Web Framework บน Fasthttp):

- **การจัดเส้นทาง (Routing & Params)**: การรับ Path Parameter (`c.Params`) และ Query Parameter (`c.Query`)
- **การรับส่งข้อมูล (JSON & BodyParser)**: การใช้ `c.JSON()` และการแปลง Request Body ด้วย `c.BodyParser()`
- **Middleware & Grouping**: การตั้งค่า Logger, CORS, Recover และการจัดกลุ่ม API Endpoint ด้วย `app.Group()`

10. การสร้าง RESTful API ด้วย Fiber (Fiber CRUD API)

สรุปการพัฒนา RESTful API ครบวงจรด้วย Fiber ร่วมกับแนวคิด Modular Package Structure:

- **โครงสร้างโปรเจกต์แบบแยก Package (Modular Structure)**: การแบ่งสถาปัตยกรรมเป็นสัดส่วน (`model`, `request`, `store`, `handler`) ให้โค้ดสะอาดและขยายง่าย
- **การจัดการ Concurrency (Thread-Safety)**: การใช้ `sync.RWMutex` (`RLock` / `Lock`) ป้องกัน Data Race เมื่อมี Goroutines เข้ามาทำงานพร้อมกัน
- **การจัดการ Slice & Memory**: เทคนิคการคัดลอก (Copy) Slice และเทคนิคการลบข้อมูลออกด้วย `append(s[:i], s[i+1:]...)`
- **การสร้าง RESTful Endpoints & Handlers**: การทำ Dependency Injection สู่ `main.go` และลงทะเบียน CRUD Routes (`GET`, `POST`, `PUT`, `DELETE`)
- **Unit Testing & Validation**: การเขียนชุดทดสอบ Unit Test แยกตามแพ็คเกจ (`store_test`, `handler_test`) และการจัดการ Error Response มาตรฐาน

11. การจัดการ Environment Variables (Env in Go)

สรุปการจัดการตัวแปรสภาพแวดล้อมเพื่อความปลอดภัยและการตั้งค่าระบบ:

- **การอ่านค่าด้วย Standard Library:** การใช้ `os.Getenv()` , `os.LookupEnv()` และการเขียน Helper Function ตรวจสอบค่าบังคับ (`log.Fatal`)
- **การใช้งานไฟล์ .env :** การโหลดค่าด้วยแพ็คเกจ `github.com/joho/godotenv` ในโหมด Development
- **การส่งค่าผ่าน CLI:** การตั้งค่าตัวแปรในระบบปฏิบัติการต่างๆ (Bash: `KEY=val` , PowerShell: `$env:KEY="val"` , CMD: `set KEY=val`)

12. การใช้งาน Middleware (Middleware in Go & Fiber)

สรุปการใช้งานและเขียน Middleware ตัวคัดกรองใน Request-Response Pipeline:

- **Built-in Middlewares ใน Fiber:** การใช้งาน `logger.New()` , `recover.New()` ป้องกัน Server ล่ม และ `cors.New()`
- **การเขียน Custom Middleware:** ตัวอย่างการทำ Secret Key Authentication และการคำนวณระยะเวลาประมวลผล (Timing)
- **Middleware & Route Grouping:** การนำ Middleware ไปประยุกต์ใช้กับเฉพาะบางกลุ่ม Endpoint ด้วย `app.Group()`

13. การสร้างเอกสาร API ด้วย Swagger (Swagger Documentation)

สรุปการสร้างเอกสาร API แบบโต้ตอบได้ (Interactive UI) ด้วย Swagger และ OpenAPI Specs:

- **แพ็คเกจและการติดตั้ง:** การใช้งาน `gofiber/swagger` และ `swaggo/swag`
- **การเขียน Swag Annotations:** การระบุ General API Info (`@title` , `@securityDefinitions.apikey`) และ Annotations กำกับแต่ละ Handler Function
- **การสร้าง Docs & การทดสอบ:** การใช้คำสั่ง `swag init` สร้างไฟล์ Specs และการเข้าใช้งานผ่าน `http://localhost:8080/swagger/index.html`

14. การใช้งาน Go database/sql & PostgreSQL (Go Database/SQL)

สรุปการใช้งานแพ็คเกจมาตรฐาน `database/sql` ร่วมกับ PostgreSQL และการจัดโครงสร้างโปรเจกต์ด้วย Repository Pattern:

- **การเชื่อมต่อ PostgreSQL:** การใช้งาน `database/sql` ร่วมกับ Driver `github.com/lib/pq` และการอ่านค่าจาก `.env`
- **การสร้าง Schema และ Constraints:** การใช้ `db.Exec()` สร้างตาราง `suppliers` และ `products` พร้อม Foreign Key Constraints
- **การทำ CRUD & Repository Pattern:** การจัด Folder Structure แบบ Clean Architecture (`config` , `models` , `repository`) และการสร้าง Constructor (`NewProductRepository`)
- **การเปรียบเทียบคำสั่ง SQL:** ข้อแตกต่างระหว่าง `db.Exec()` , `db.Query()` , และ `db.QueryRow()`
- **การจัดการ Pointer & NULL:** การตั้งค่าและแสดงผลฟิลด์ Pointer (`*int` , `*string`) ด้วย `fmt.Printf` อย่างปลอดภัย

- **การแก้ปัญหาพบบ่อย (Troubleshooting):** การจัดการ `rows.Err()` , แก้ไข Foreign Key Violation (Code `23503`), และเรื่อง Variable Scope / Type Safety ใน Go

15. การจัดการ Database Transaction (Database Transaction in Go)

สรุปการใช้งาน Database Transaction เพื่อรักษารวมความถูกต้องของข้อมูลตามหลัก ACID:

- **หลักการ ACID:** ทำความเข้าใจ Atomicity, Consistency, Isolation, และ Durability
- **Safe Transaction Pattern:** เทคนิคการใช้ `db.Begin()` , `defer tx.Rollback()` , และ `tx.Commit()` เพื่อป้องกันข้อมูลซ้ำ
- **ตัวอย่างระบบจริง:** โค้ดตัวอย่างระบบโอนเงินข้ามบัญชี และระบบสร้าง Order พร้อมหัก Stock สินค้า
- **การจัดการ Deadlock & Retry:** สาเหตุการเกิด Deadlock (Code `40P01`), เทคนิค Consistent Lock Ordering และการทำ Retry Mechanism ผัง Application
- **การกำหนด Isolation Level:** การใช้ `db.BeginTx()` กำหนด Isolation Level และการใช้ร่วมกับ `context.Context`

16. การใช้งาน Stored Procedure & Function (Stored Procedure in Go)

สรุปการสร้าง Stored Procedure และ Function ใน PostgreSQL ร่วมกับการเรียกใช้งานจากภาษา Go:

- **Procedure vs Function:** ความแตกต่างของ `CREATE FUNCTION` (คืนค่า) และ `CREATE PROCEDURE` (ควบคุม Transaction)
- **การเรียกใช้งานใน Go:** เทคนิคการใช้ `SELECT function_name()` ดึงค่า Scalar/Table และการใช้ `CALL procedure_name()` ส่งรันด้วย `db.Exec()`
- **ตัวอย่างโค้ดระบบจริง:** Function คำนวณ VAT, Function ดึงรายการสินค้าแบบ Table และ Procedure ย้ายสินค้าข้าม Supplier
- **ข้อดีและข้อเสีย:** เปรียบเทียบ Performance, Security, Vendor Lock-in, และความยากง่ายในการ Maintenance/Testing

คำแนะนำ คลิกที่หัวข้อด้านบนเพื่อไปยังบทเรียนภายในเอกสาร PDF ฉบับนี้ได้เลยครับ!

เปรียบเทียบ Array vs Slice ในภาษา Go (Golang)

1. ตารางเปรียบเทียบข้อแตกต่าง (Comparison Table)

หัวข้อ (Feature)	Array	Slice
ขนาด (Size)	คงที่ (Fixed size) ต้องระบุขนาดตั้งแต่แรก	ยืดหยุ่น ขยาย/ลดขนาดได้ (Dynamic size)
การประกาศ (Syntax)	<code>[5]int{1, 2, 3, 4, 5}</code> (ระบุขนาดใน <code>[]</code>)	<code>[]int{1, 2, 3, 4, 5}</code> (ไม่ระบุขนาดใน <code>[]</code>)
ชนิดข้อมูล (Type)	ขนาดเป็นส่วนหนึ่งของ Type (เช่น <code>[3]int</code> กับ <code>[5]int</code> ถือเป็นคนละชนิดข้อมูล)	เป็นชนิดข้อมูลเดียวกันคือ <code>[]int</code> (ไม่ว่าจะมีความยาวเท่าใด)
การส่งค่าเข้าฟังก์ชัน (Passing Mechanism)	Pass by Value (คัดลอกข้อมูลอาร์เรย์ทั้งหมดไปใหม่)	Pass by Reference / Header (ส่งโครงสร้างที่ชี้ไปยัง Underlying Array เดิม)
โครงสร้างภายใน (Internal Structure)	เก็บข้อมูลเรียงต่อกันโดยตรงในหน่วยความจำ	มี 3 ส่วน: Pointer, Length (<code>len</code>), Capacity (<code>cap</code>)
การเพิ่มข้อมูล	ไม่สามารถใช้ <code>append()</code> เพิ่มข้อมูลได้	ใช้ <code>append()</code> เพิ่มข้อมูลได้อย่างยืดหยุ่น
การแปลงข้อมูล (Conversion)	แปลงเป็น Slice ได้ง่ายด้วย Slicing <code>arr[:]</code>	แปลงเป็น Array ได้ตั้งแต่ Go 1.20+ (<code>[N]T(slice)</code>) หรือ Pointer (<code>*[N]T(slice)</code>)

2. โครงสร้างภายในของ Slice (Slice Header)

Slice ใน Go ทำหน้าที่เป็นตัวห่อหุ้ม (Wrapper/Abstraction) ซ่อนอยู่บน **Array** เบื้องหลัง (**Underlying Array**) ซึ่งประกอบด้วย 3 필ด์หลัก:

- Pointer**: ตัวชี้ไปยังตำแหน่งแรกของ Underlying Array ที่ Slice นั้นอ้างอิงอยู่
- Length (len)**: จำนวนสมาชิกที่มีอยู่ใน Slice ณ ปัจจุบัน
- Capacity (cap)**: ความจุสูงสุดที่ Slice สามารถรับได้ โดยนับจากตำแหน่ง Pointer ไปจนถึงตัวสุดท้ายของ Underlying Array

3. ตัวอย่างโค้ดประกอบการใช้งาน (Code Examples)

ตัวอย่าง 1: Array

```
package main

import "fmt"

func main() {
    // ประกาศ Array ขนาด 3
    var arr [3]int = [3]int{10, 20, 30}
    fmt.Println(arr) // [10 20 30]

    // arr = append(arr, 40) ❌ คอมไพล์ไม่ผ่าน! ไม่สามารถใช้ append กับ Array ได้
}
```

ตัวอย่าง 2: Slice

```
package main

import "fmt"

func main() {
    // ประกาศ Slice (ไม่ระบุขนาดใน [])
    slice := []int{10, 20, 30}
    fmt.Println(slice) // [10 20 30]

    // เพิ่มสมาชิกด้วย append()
    slice = append(slice, 40)
    fmt.Println(slice) // [10 20 30 40]
    fmt.Println(len(slice)) // 4
    fmt.Println(cap(slice)) // 6 (Go จะเพิ่ม Capacity ให้อัตโนมัติเมื่อข้อมูลเต็ม)
}
```


ตัวอย่าง 3: การตัดแบ่ง (Slicing) และค่า `len` / `cap`

```
package main

import "fmt"

func main() {
    mySlice := []int{1, 2, 3, 4, 5}
    fmt.Println(mySlice)      // [1 2 3 4 5]
    fmt.Println(len(mySlice)) // 5
    fmt.Println(cap(mySlice)) // 5

    // ตัดเอาเฉพาะ Index 1 ถึงก่อน Index 3
    subSlice := mySlice[1:3]
    fmt.Println(subSlice)     // [2 3]
    fmt.Println(len(subSlice)) // 2 (มีสมาชิก 2 ตัว คือ 2 และ 3)
    fmt.Println(cap(subSlice)) // 4 (นับจาก Index 1 คือค่า 2 ไปจนถึงตัวสุดท้าย 5)
}
```

4. การแปลงข้อมูลระหว่าง Array และ Slice (Array & Slice Conversion)

4.1 การแปลง Array เป็น Slice (Array to Slice)

สามารถใช้ **Slicing Expression** (`arr[:]` หรือ `arr[low:high]`) เพื่อแปลง Array ให้กลายเป็น Slice ได้ทันที

*หมายเหตุ Slice ที่ได้จะชี้ไปยัง **Underlying Array** ตัวเดิม ดังนั้น หากมีการแก้ไขสมาชิกใน Slice ข้อมูลใน Array ต้นทางจะเปลี่ยนแปลงตามไปด้วย*

```

package main

import "fmt"

func main() {
    arr := [5]int{10, 20, 30, 40, 50}

    // แปลง Array เป็น Slice
    slice := arr[:]

    // แก้ไขข้อมูลผ่าน Slice
    slice[0] = 99

    fmt.Println("Array:", arr) // Array: [99 20 30 40 50] (ค่าเปลี่ยนตาม)
    fmt.Println("Slice:", slice) // Slice: [99 20 30 40 50]
}

```

4.2 การแปลง Slice เป็น Array (Slice to Array)

1) การแปลงเป็น Array โดยตรง ([N]T(slice)) - Go 1.20+

ตั้งแต่ **Go 1.20** เป็นต้นไป เราสามารถแปลง Slice เป็น Array ได้โดยตรงด้วยไวยากรณ์ `[N]T(slice)` โดยการแปลงวิธีนี้จะเป็นการ **คัดลอกค่า (Copy / Pass by Value)** ไปสร้าง Array ชุดใหม่

```

package main

import "fmt"

func main() {
    slice := []int{1, 2, 3, 4, 5}

    // แปลง Slice เป็น Array ขนาด 3
    arr := [3]int(slice[:3])

    arr[0] = 999
    fmt.Println("Array:", arr) // Array: [999 2 3]
    fmt.Println("Slice:", slice) // Slice: [1 2 3 4 5] ( Slice เดิมไม่เปลี่ยน)
}

```

คำเตือน หากขนาดของ Array ที่ต้องการแปลง มีขนาดมากกว่าความยาว (`Len`) ของ Slice จะทำให้เกิด **Runtime Panic** (เช่น `[10]int(slice)` ทั้งที่ `slice` มี `len` เป็น 5)

2) การแปลงเป็น Array Pointer ((*[N]T)(slice)) - Go 1.17+

ตั้งแต่ **Go 1.17** เป็นต้นไป สามารถแปลง Slice ให้เป็น Pointer ของ Array ได้ ไวยากรณ์นี้ไม่ได้ทำการคัดลอกข้อมูลใหม่ แต่ชี้ไปยังหน่วยความจำเดิม

```
package main

import "fmt"

func main() {
    slice := []int{1, 2, 3, 4, 5}

    // แปลง Slice เป็น Pointer ของ Array ขนาด 5
    arrPtr := (*[5]int)(slice)

    fmt.Println("Array Pointer:", *arrPtr) // [1 2 3 4 5]
}
```

[กลับไปสารบัญ](#)

การใช้งาน Map ในภาษา Go (Map in Go)

Map คือ โครงสร้างข้อมูลที่เก็บข้อมูลในรูปแบบ **Key-Value Pairs** (เทียบเท่ากับ Dictionary ใน Python, Hash Table ใน C++ หรือ Object/Map ใน JavaScript) โดย Key แต่ละตัวจะต้องเป็นประเภทที่เปรียบเทียบกันได้ (comparable) และไม่ซ้ำกัน

1. ความสำคัญของ Map (Why Map is Important)

1. ค้นหาข้อมูลรวดเร็วระดับ $O(1)$ (Fast Lookup Time Complexity):

- เมื่อเทียบกับ Slice ที่ต้องวนลูปหาข้อมูลแบบ $O(n)$ การค้นหาด้วย Key ใน Map ใช้เวลาคงที่ $O(1)$ ไม่ว่าข้อมูลจะมีจำนวนหลักแสนหรือหลักล้านรายการ

2. ความยืดหยุ่นในการใช้ Key (Flexible Key Types):

- ต่างจาก Slice ที่ต้องอ้างอิงด้วย Index ตัวเลข 0, 1, 2... เท่านั้น แต่ Map สามารถใช้ `string`, `int`, `struct` หรือประเภทข้อมูลอื่นที่เปรียบเทียบกันได้มาเป็น Key

3. ใช้สร้างโครงสร้างข้อมูลแบบ Custom (Emulating Sets & In-memory Cache):

- ใช้ทำ Caching, Lookup Table, นับความถี่ข้อมูล (Frequency Counter), จัดกลุ่มข้อมูล (Group By), และจำลอง Set ป้องกันค่าซ้ำ

2. ลักษณะสำคัญและข้อควรระวัง (Characteristics & Pitfalls)

- Unordered:** ลำดับข้อมูลใน Map ไม่แน่นอน การวนลูปด้วย `for key, val := range m` แต่ละครั้งอาจได้ลำดับไม่เหมือนกัน
- Reference Type:** Map เป็น Reference Type เมื่อส่งเข้าฟังก์ชัน จะเป็นการส่ง Pointer อ้างอิงไปยัง memory ชุดเดิม (การแก้ไขภายในฟังก์ชันจะกระทบต้นทาง)
- Zero Value คือ nil :**
 - ประกาศ `var m map[string]int` จะได้ค่าเป็น `nil`
 - สามารถอ่านค่าจาก `nil map` ได้ (คืนค่า Zero Value ของชนิดนั้น) แต่ ห้ามเขียน/เพิ่มข้อมูลลง `nil map` ทันที เพราะจะเกิด `panic: assignment to entry in nil map`
 - ต้องสร้างผ่าน `make(map[KeyType]ValueType)` หรือ Map Literal (`map[string]int{}`) ก่อนเสมอ
- Not Thread-Safe:**
 - Map ใน Go ไม่รองรับการเขียนแบบ Concurrent หากหลาย Goroutine เขียน Map ตัวเดียวกันพร้อมกัน จะเกิด `fatal error: concurrent map writes`
 - ทางแก้ไข: ต้องใช้ `sync.RWMutex` คอยล็อกก่อนอ่าน/เขียน หรือใช้ `sync.Map`

3. ตัวอย่างการใช้งานพื้นฐาน (Basic Operations)

```
package main

import "fmt"

func main() {
    // 1. การสร้าง Map ด้วย make()
    myMap := make(map[string]int)

    // การเพิ่มและอัปเดตข้อมูล (Insert & Update)
    myMap["apple"] = 3
    myMap["banana"] = 6
    myMap["banana"] = 12 // อัปเดตค่าเดิม

    // 2. การเข้าถึงข้อมูลด้วย Key (Access Value)
    fmt.Println("ค่าของ banana:", myMap["banana"]) // 12

    // 3. การลบข้อมูลด้วย delete() (Delete Key)
    delete(myMap, "apple")

    // 4. การตรวจสอบว่ามี Key อยู่หรือไม่ (Comma-ok Idiom)
    val, ok := myMap["apple"]
    if ok {
        fmt.Println("พบ apple:", val)
    } else {
        fmt.Println("ไม่พบ apple ใน Map") // ทำงานบรรทัดนี้
    }
}
```

4. ตัวอย่างการใช้งานในระบบจริง (Real-world Code Examples)

ตัวอย่างที่ 1: การจำลอง Set เพื่อเช็คและป้องกันค่าซ้ำ (Set Emulation)

ภาษา Go ไม่มีชนิดข้อมูล Set ในตัว เรานิยมใช้ `map[KeyType]bool` หรือ `map[KeyType]struct{}` ในการเก็บค่าไม่ซ้ำ:

```

package main

import "fmt"

func removeDuplicates(items []string) []string {
    seen := make(map[string]bool)
    result := []string{}

    for _, item := range items {
        if !seen[item] { // ถ้ายังไม่เคยเจอใน Map
            seen[item] = true
            result = append(result, item)
        }
    }
    return result
}

func main() {
    emails := []string{"a@test.com", "b@test.com", "a@test.com", "c@test.com"}
    uniqueEmails := removeDuplicates(emails)
    fmt.Println("Emails แบบไม่ซ้ำ:", uniqueEmails) // [a@test.com b@test.com
c@test.com]
}

```

ตัวอย่างที่ 2: การนับความถี่ของข้อมูล (Frequency Counter)

ใช้นับจำนวนสถิติ เช่น นับจำนวนคำ หรือนับสถิติการใช้งาน:

```

package main

import "fmt"

func countWords(text []string) map[string]int {
    counts := make(map[string]int)
    for _, word := range text {
        counts[word]++ // หากไม่มี Key ค่าจะเริ่มจาก Zero Value (0) แล้ว +1
    }
    return counts
}

func main() {
    words := []string{"go", "fiber", "go", "gin", "go", "fiber"}
    fmt.Println("จำนวนคำ:", countWords(words)) // map[fiber:2 gin:1 go:3]
}

```

ตัวอย่างที่ 3: การทำ In-Memory Lookup Index / Cache

ค้นหาข้อมูลวัตถุด้วย Unique ID ระดับความเร็ว $O(1)$:

```
type User struct {
    ID    int
    Name  string
}

// แปลง Slice เป็น Map เพื่อให้ค้นหาด้วย ID ได้รวดเร็ว  $O(1)$ 
func buildUserIndex(users []User) map[int]User {
    userMap := make(map[int]User)
    for _, u := range users {
        userMap[u.ID] = u
    }
    return userMap
}
```

ตัวอย่างที่ 4: การจัดกลุ่มข้อมูล (Group By)

```
type Product struct {
    Name      string
    Category  string
}

func groupByCategory(products []Product) map[string][]Product {
    grouped := make(map[string][]Product)
    for _, p := range products {
        grouped[p.Category] = append(grouped[p.Category], p)
    }
    return grouped
}
```

5. ตารางสรุปคำสั่งคำนวณที่สำคัญ

การทำงาน (Operation)	ไวยากรณ์ (Syntax)	คำอธิบาย
สร้าง Map	<code>make(map[K]V)</code>	สร้าง Map พร้อมใช้งาน (ป้องกัน nil panic)
เพิ่ม/อัปเดต	<code>m[key] = value</code>	กำหนดค่าให้ผูกกับ Key
อ่านค่า	<code>val := m[key]</code>	คืนค่า Value หรือ Zero Value หากไม่มี Key

การทำงาน (Operation)	ไวยากรณ์ (Syntax)	คำอธิบาย
เช็คการมีอยู่	<code>val, ok := m[key]</code>	Comma-ok idiom (<code>ok</code> เป็น <code>true</code> เมื่อมี Key)
ลบ Key	<code>delete(m, key)</code>	ลบ Key ออกจาก Map
นับจำนวน Key	<code>len(m)</code>	คืนค่าจำนวน Key ทั้งหมดใน Map

[กลับไปสารบัญ](#)

การใช้งาน Struct ในภาษา Go (Struct in Go)

Struct คือ โครงสร้างข้อมูลแบบกำหนดเอง (Custom Data Type) ที่เกิดจากการรวมกลุ่มของตัวแปรประเภทต่างๆ (Fields) เข้าด้วยกันเป็นชนิดข้อมูลใหม่ เหมาะสำหรับใช้แทนวัตถุที่มีหลายคุณสมบัติในโลกจริง (เช่น ข้อมูลนิสิตนักศึกษา ข้อมูลที่อยู่ ฯลฯ)

1. การประกาศและสร้าง Struct (Declaration & Instantiation)

1.1 การประกาศ Struct พื้นฐาน

```
type Student struct {  
    Name    string  
    Age     int  
    Weight  float32  
    Height  int  
    Grade   string  
}
```

1.2 Nested Struct (Struct ซ้อน Struct)

ในภาษา Go เราสามารถนำ Struct หนึ่งไปเป็น Field ภายในอีก Struct หนึ่งได้ เพื่อจัดกลุ่มข้อมูลให้เป็นระเบียบมากยิ่งขึ้น

```
type Address struct {  
    City      string  
    ZIPCode   string  
    Country   string  
}  
  
type Student struct {  
    Name      string  
    Age       int  
    Weight    float32  
    Height    int  
    Grade     string  
    Address   Address // Nested Struct  
}
```

2. ตัวอย่างการใช้งาน (Code Examples)

2.1 การใช้งาน Struct ร่วมกับ Map และการเข้าถึง Nested Struct

เราสามารถสร้าง `Slice` หรือ `Map` ที่เก็บข้อมูลเป็น Struct เพื่อประยุกต์ใช้งานจริงได้

```
package main

import "fmt"

type Address struct {
    City    string
    ZIPCode string
    Country string
}

type Student struct {
    Name      string
    Age       int
    Weight    float32
    Height    int
    Grade     string
    Address   Address // Nested Struct
}

func main() {
    // 1. สร้าง Map ที่เก็บ Key เป็น string และ Value เป็น Student Struct
    students := make(map[string]Student)

    // 2. กำหนดค่าข้อมูลนิสิตพร้อมข้อมูล Address ที่ซ่อนอยู่ภายใน
    students["st001"] = Student{
        Name:    "John",
        Age:     20,
        Weight:  71.5,
        Height:  175,
        Grade:   "A",
        Address: Address{
            City:    "New York",
            ZIPCode: "10001",
            Country: "USA",
        },
    }

    students["st002"] = Student{
        Name:    "Adam",
        Age:     30,
        Weight:  75.5,
        Height:  180,
        Grade:   "B",
        Address: Address{
            City:    "Chicago",
            ZIPCode: "60601",
            Country: "USA",
        },
    }
}
```

```

    },
}

// 3. วนลูปอ่านข้อมูลใน Map และเข้าถึง Nested Struct Fields
for id, student := range students {
    fmt.Println("ID:", id)
    fmt.Println("Name:", student.Name)
    fmt.Println("Age:", student.Age)
    fmt.Println("Grade:", student.Grade)
    // เข้าถึงข้อมูล Nested Struct ด้วย Dot Notation ซ้อนกัน
    fmt.Println("City:", student.Address.City)
    fmt.Println("ZIPCode:", student.Address.ZIPCode)
    fmt.Println("Country:", student.Address.Country)
    fmt.Println("-----")
}
}

```

3. สรุปจุดสำคัญของ Struct

หัวข้อ	คำอธิบาย
Field Access	เข้าถึงหรือแก้ไขค่าใน Struct ใช้เครื่องหมายจุด (.) เช่น <code>student.Name</code>
Nested Field Access	เข้าถึง Struct ที่ซ้อนอยู่ด้านใน ใช้จุดซ้อนจุด เช่น <code>student.Address.City</code>
Zero Value	หากไม่ได้ระบุค่าให้ Field ใด ค่าเริ่มต้นจะเป็น Zero Value ของชนิดข้อมูลนั้น (เช่น <code>0</code> , <code>""</code> , <code>false</code>)
Pass by Value	โดยพื้นฐานการส่ง Struct เข้าฟังก์ชันจะเป็นการ Pass by Value (คัดลอกข้อมูลใหม่) หากต้องการให้แก้ไขข้อมูลเดิมได้ต้องใช้ Pointer (<code>*Student</code>)

[กลับไปสารบัญ](#)

เปรียบเทียบ Function vs Method ในภาษา Go (Golang)

1. ตารางเปรียบเทียบข้อแตกต่าง (Comparison Table)

หัวข้อ (Feature)	Function (ฟังก์ชัน)	Method (เมธอด)
การผูกกับ Type	เป็นอิสระ ไม่ผูกติดกับ Type หรือ Struct ใดๆ	ต้องผูกติดกับ Type (Receiver Type) เช่น Struct
การเรียกใช้งาน (Invocation)	เรียกผ่านชื่อฟังก์ชันโดยตรง เช่น <code>sum(10, 20)</code>	เรียกผ่านอินสแตนซ์ของ Type (Dot Notation) เช่น <code>student.FullName()</code>
Syntax การประกาศ	<code>func Name(params) Return Type</code>	<code>func (r ReceiverType) Name(params) Return Type</code>
Receiver	ไม่มี Receiver	มี Receiver (แบ่งเป็น Value Receiver และ Pointer Receiver)
การเข้าถึงข้อมูลภายใน Struct	ต้องส่ง Struct เข้ามาทาง Parameter	สามารถเข้าถึงฟิลด์ของ Receiver ได้โดยตรงผ่านตัวแปร Receiver
วัตถุประสงค์ (Use Case)	ใช้สำหรับประมวลผลทั่วไป (Utility/Generic logic)	ใช้สำหรับแสดงพฤติกรรม (Behavior) ของ Object/Type นั้นๆ

2. โครงสร้างและการประกาศ (Syntax & Structure)

2.1 Function (ฟังก์ชันทั่วไป)

ฟังก์ชันคือบล็อกโค้ดอิสระ รับอินพุต (Parameters) ประมวลผล และส่งคืนค่า (Return value) โดยไม่ต้องมี Receiver

```
func FullName(firstName string, lastName string) string {  
    return firstName + " " + lastName  
}
```

2.2 Method (เมธอด)

เมธอดคือฟังก์ชันที่มี **Receiver Argument** อยู่ระหว่างคีย์เวิร์ด `func` และชื่อเมธอด

```

type Student struct {
    FirstName string
    LastName  string
}

// Receiver (s Student) ผูกกับ Student struct
func (s Student) FullName() string {
    return s.FirstName + " " + s.LastName
}

```

3. Value Receiver vs Pointer Receiver ใน Method

ใน Go การสร้าง Method บน Receiver สามารถแบ่งออกเป็น 2 ชนิดหลัก:

3.1 Value Receiver `func (s Student)`

- เป็นการส่งค่าแบบ **Pass by Value** (คัดลอกข้อมูลอินสแตนซ์เดิมมาใหม่)
- การแก้ไขฟิลด์ภายในเมธอด **จะไม่มีผล** ต่ออินสแตนซ์เดิมภายนอก
- เหมาะสำหรับเมธอดที่อ่านค่าอย่างเดียว (Read-only) หรือ Struct ที่มีขนาดเล็ก

```

func (s Student) ChangeName(newName string) {
    s.FirstName = newName // ❌ เปลี่ยนเฉพาะสำเนา ตัวจริงไม่เปลี่ยน
}

```

3.2 Pointer Receiver `func (s *Student)`

- เป็นการส่งตัวชี้ตำแหน่งหน่วยความจำ **Pass by Reference/Pointer**
- การแก้ไขฟิลด์ภายในเมธอด **จะมีผล** ต่ออินสแตนซ์เดิมภายนอกทันที
- เหมาะสำหรับเมธอดที่ต้องการแก้ไขข้อมูล (Mutate State) หรือ Struct ที่มีขนาดใหญ่เพื่อหลีกเลี่ยงการคัดลอกข้อมูลในหน่วยความจำ

```

func (s *Student) UpdateName(newName string) {
    s.FirstName = newName // ✅ เปลี่ยนแปลงข้อมูลในอินสแตนซ์เดิมจริง
}

```

4. ตัวอย่างโค้ดเปรียบเทียบการใช้งาน (Code Example)

```
package main

import "fmt"

type Student struct {
    FirstName string
    LastName  string
}

// 1. Function: ส่งค่าเข้ามาทาง Parameter
func GetFullNameFunc(s Student) string {
    return s.FirstName + " " + s.LastName
}

// 2. Method (Value Receiver): อ่านค่าผ่าน Receiver
func (s Student) FullName() string {
    return s.FirstName + " " + s.LastName
}

// 3. Method (Pointer Receiver): แก้ไขค่าเดิมผ่าน Pointer Receiver
func (s *Student) SetFirstName(newName string) {
    s.FirstName = newName
}

func main() {
    student := Student{
        FirstName: "John",
        LastName:  "Doe",
    }

    // เรียกใช้งาน Function
    fmt.Println("Func Output:", GetFullNameFunc(student))

    // เรียกใช้งาน Method (Value Receiver)
    fmt.Println("Method Output:", student.FullName())

    // เรียกใช้งาน Method (Pointer Receiver) เพื่ออัปเดตชื่อ
    student.SetFirstName("Jane")
    fmt.Println("Updated Method Output:", student.FullName())
}
```

5. สรุปเมื่อไหร่ควรใช้ Function หรือ Method

1. ใช้ Function เมื่อ:

- เป็นการประมวลผลทั่วไป เช่น คำนวณคณิตศาสตร์ (`math.Max`), จัดการ String (`strings.ToUpper`)
- ไม่เกี่ยวข้องกับ State หรือคุณสมบัติเฉพาะของ Struct ชนิดใดชนิดหนึ่ง

2. ใช้ Method เมื่อ:

- ต้องการกำหนดพฤติกรรม (Behavior) ให้กับ Struct หรือ Custom Type นั้นๆ
- ต้องการทำให้โค้ดอ่านง่ายในรูปแบบ Object-Oriented Style (เช่น `student.FullName()` , `user.Save()`)
- ต้องการเตรียมไว้รองรับการทำ **Interface** ใน Go

การใช้งาน Interface ในภาษา Go (Interface in Go)

Interface คือ ชนิดข้อมูลชนิดหนึ่งใน Go ที่ใช้กำหนด ข้อตกลงหรือสัญญา (Contract/Behavior) ว่า Type ที่ทำตาม Interface นี้จะต้องมี Method อะไรบ้าง (Signature ไหน) โดยใน Go จะใช้วิธี **Implicit Implementation** (ไม่ต้องใช้คีย์เวิร์ด `implements`) ชนิดข้อมูลใดที่มี Method ครบตามที่ Interface กำหนด จะถือว่าเป็น Interface นั้นทันที!

1. การประกาศและแนวคิดพื้นฐาน (Syntax & Concept)

1.1 การประกาศ Interface

```
type Shape interface {  
    Area() float64  
}
```

1.2 Implicit Implementation (ทำตามสัญญาโดยอัตโนมัติ)

ใน Go หาก Struct หรือ Custom Type ใดมี Method ที่ชื่อ และ Signature (พารามิเตอร์และชนิดค่าที่ส่งกลับ) ตรงกับที่ Interface กำหนด จะถือว่าเป็น Interface นั้นทันทีโดยไม่ต้องระบุคีย์เวิร์ดใดๆ เพิ่มเติม

```
// Circle มี Method Area() float64 -> ถือว่าเป็น Shape โดยอัตโนมัติ  
type Circle struct {  
    R float64  
}  
  
func (c Circle) Area() float64 {  
    return 3.14 * c.R * c.R  
}  
  
// Square มี Method Area() float64 -> ถือว่าเป็น Shape เช่นกัน  
type Square struct {  
    Side float64  
}  
  
func (s Square) Area() float64 {  
    return s.Side * s.Side  
}
```

2. พหุรูปร่าง (Polymorphism) ด้วย Interface

ประโยชน์หลักของ Interface คือทำให้เราสามารถเขียนฟังก์ชันที่รับ Parameter เป็น Interface เดียวกัน แต่ส่ง Struct ต่างชนิดกันที่มีพฤติกรรมตรงตาม Interface เข้ามาประมวลผลได้

```
// ฟังก์ชัน printArea รับ Parameter เป็น Shape (รับได้ทั้ง Circle, Square หรือชนิดข้อมูลอื่นที่เป็น Shape)
func printArea(s Shape) {
    fmt.Println(s.Area())
}
```

3. ตัวอย่างโค้ดประกอบการใช้งาน (Code Examples)

3.1 ตัวอย่างที่ 1: Interface คำนวณพื้นที่รูปทรงเรขาคณิต (Shape & Area)

```
package main

import "fmt"

// 1. ประกาศ Interface
type Shape interface {
    Area() float64
}

// 2. Struct ที่ 1: Circle (วงกลม)
type Circle struct {
    R float64
}

func (c Circle) Area() float64 {
    return 3.14 * c.R * c.R
}

// 3. Struct ที่ 2: Square (สี่เหลี่ยมจัตุรัส)
type Square struct {
    Side float64
}

func (s Square) Area() float64 {
    return s.Side * s.Side
}

// 4. ฟังก์ชันที่รับ Interface เป็น Parameter (Polymorphism)
func printArea(s Shape) {
    fmt.Println(s.Area())
}

func main() {
    circle := Circle{R: 5}
    square := Square{Side: 5}

    // ทั้ง circle และ square สามารถส่งเข้า printArea() ได้เพราะทั้งคู่มี Method Area()
    float64
    printArea(circle) // Output: 78.5 (3.14 * 5 * 5)
    printArea(square) // Output: 25 (5 * 5)
}
```

3.2 ตัวอย่างที่ 2: Interface สำหรับการส่งเสียงพูด (Speaker & Speak)

```
package main

import "fmt"

// 1. ประกาศ Interface
type Speaker interface {
    Speak() string
}

// 2. Struct ที่ 1: Person
type Person struct {
    Name string
}

func (p Person) Speak() string {
    return "Hello, my name is " + p.Name
}

// 3. Struct ที่ 2: Dog
type Dog struct {
    Name string
}

func (d Dog) Speak() string {
    return "Woof! My name is " + d.Name
}

// 4. ฟังก์ชันที่รับ Interface เป็น Parameter
func makeSound(s Speaker) {
    fmt.Println(s.Speak())
}

func main() {
    person := Person{Name: "John"}
    dog := Dog{Name: "Buddy"}

    makeSound(person) // Output: Hello, my name is John
    makeSound(dog)    // Output: Woof! My name is Buddy
}
```

4. คุณสมบัติและแนวคิดสำคัญของ Interface ใน Go

หัวข้อ	คำอธิบาย
Implicit Implementation	ไม่ต้องใช้คีย์เวิร์ด <code>implements</code> แค่สร้าง Method ให้ตรง Signature ก็ถือว่าทำตามสัญญาแล้ว
Decoupling & Flexibility	ช่วยลดความยึดติดกันของโค้ด (Loose Coupling) เพิ่มความยืดหยุ่น และเอื้อต่อการเขียน Unit Test / Mocking
Empty Interface (<code>interface{}</code> / <code>any</code>)	Interface ที่ไม่มี Method กำหนด สามารถรับค่าข้อมูลได้ทุกชนิดในภาษา Go (ใช้อักษรย่อ <code>any</code> ได้ตั้งแต่ Go 1.18+)
Type Assertion	เทคนิคการตรวจสอบและแปลงข้อมูลจาก Interface กลับเป็น Concrete Type เดิม เช่น <code>c, ok := s.(Circle)</code>

[กลับไปสารบัญ](#)

การใช้งาน Pointer ในภาษา Go (Pointer in Go)

Pointer (พอยน์เตอร์) คือตัวแปรประเภทหนึ่งในภาษา Go ที่ใช้เก็บ ตำแหน่งที่อยู่บนหน่วยความจำ (Memory Address) ของตัวแปรอื่น แทนที่จะเก็บค่าข้อมูลโดยตรง การใช้งาน Pointer ช่วยให้เราสามารถอ้างอิงและแก้ไขข้อมูลต้นทางได้โดยตรงโดยไม่ต้องคัดลอกข้อมูล (Pass by Reference)

1. สัญลักษณ์สำคัญของ Pointer (Key Operators)

เครื่องหมาย	ชื่อเรียก	คำอธิบายการใช้งาน	ตัวอย่าง
&	Address-of Operator	ใช้ดึง ตำแหน่งที่อยู่หน่วยความจำ (Memory Address) ของตัวแปร	p := &x
*	Dereference Operator / Pointer Type	1. ใช้ระบุชนิดตัวแปร Pointer: เช่น v *int 2. ใช้ดึง/แก้ไข ค่าที่อยู่ในที่อยู่นั้น: เช่น *p = 77	var p *int *p = 77

หมายเหตุ **Automatic Dereferencing** ใน Go: สำหรับ Struct Pointer เช่น e *Employee เราสามารถเข้าถึงฟิลด์ได้โดยตรงด้วย e.Salary โดยไม่จำเป็นต้องเขียน (*e).Salary เพราะ Go Compiler จะจัดการดึงค่าจาก Pointer ให้อัตโนมัติ

2. การทำงานของ Pointer ในหน่วยความจำ (Memory Concept)

เมื่อเรากำหนด y := &x ตัวแปร y จะเก็บตำแหน่งหน่วยความจำของ x (เช่น 0xc0000a2008)

[ตัวแปร x] ---> ตำแหน่งในหน่วยความจำ: 0xc0000a2008 ---> ค่าข้างใน: 50
[ตัวแปร y] ---> เก็บตำแหน่ง: 0xc0000a2008 (ชี้ไปยัง x)

เมื่อสั่งคำสั่ง *y = 77 คอมไพเลอร์จะเดินตามที่อยู่ใน y ไปยังตำแหน่ง 0xc0000a2008 แล้วเปลี่ยนค่าข้างในกล่องนั้นเป็น 77 ส่งผลให้เมื่ออ่านค่าของ x และ *y จะได้ค่า 77 เช่นเดียวกัน

3. ตัวอย่างโค้ดประกอบการใช้งาน (Code Examples)

3.1 ตัวอย่างที่ 1: การใช้ Pointer กับชนิดข้อมูลพื้นฐาน (int)

```
package main

import "fmt"

// 1. ฟังก์ชันรับ Parameter เป็น Pointer (Pass by Reference)
func changeValue(v *int) {
    *v = 50 // เปลี่ยนค่าในตำแหน่งหน่วยความจำที่ v ชี้อยู่ให้เป็น 50
}

func main() {
    x := 3
    fmt.Println("x เริ่มต้น:", x) // Output: 3

    // 2. ส่ง Memory Address ของ x (&x) เข้าไปในฟังก์ชัน
    changeValue(&x)
    fmt.Println("x หลังเรียก changeValue:", x) // Output: 50

    // 3. ประกาศตัวแปร Pointer y ชี้ไปยังตำแหน่งของ x
    y := &x
    fmt.Println("ตำแหน่งที่อยู่หน่วยความจำของ x (y):", y) // Output: 0xc000...
    (Memory Address)
    fmt.Println("ค่าที่ y ชี้อยู่ (*y):", *y) // Output: 50

    // 4. แก้ไขค่าผ่าน Pointer y (Dereferencing)
    *y = 77
    fmt.Println("x หลังถูกเปลี่ยนผ่าน *y:", x) // Output: 77
    fmt.Println("ค่าที่ y ชี้อยู่ (*y):", *y) // Output: 77
}
```

3.2 ตัวอย่างที่ 2: การใช้ Pointer ร่วมกับ Struct (Employee)

```
package main

import "fmt"

type Employee struct {
    Name    string
    Age     int
    Salary  float64
}

// ฟังก์ชันรับ Struct Pointer e *Employee เพื่อแก้ไขเงินเดือนของ Employee ดันทาง
func giveRaise(e *Employee, percent float64) {
    // ใช้ e.Salary ได้เลยโดยไม่ต้องใช้ (*e).Salary (Automatic Dereferencing)
    e.Salary += e.Salary * (percent / 100)
}

func main() {
    emp := Employee{
        Name:    "John",
        Age:     32,
        Salary:  67500,
    }

    fmt.Println("Before:", emp) // Output: Before: {John 32 67500}

    // ส่ง Memory Address ของ emp (&emp) เพื่อให้ฟังก์ชันปรับแก้ไขเงินเดือนดันทาง
    giveRaise(&emp, 15)

    fmt.Println("After:", emp) // Output: After: {John 32 77625}
}
```

4. ประโยชน์และช่วงเวลาที่เราควรเลือกใช้ Pointer

1. ต้องการให้ฟังก์ชันแก้ไขข้อมูลต้นทางได้ (**Mutate State**): เพื่อให้การเปลี่ยนแปลงข้อมูลภายในฟังก์ชันส่งผลกระทบต่อแปรหรือ Struct ภายนอก
2. ประสิทธิภาพและการประหยัดหน่วยความจำ (**Performance Optimization**): หลีกเลี่ยงการคัดลอกข้อมูลขนาดใหญ่ (เช่น Struct ที่มีหลายฟิลด์) เมื่อต้องส่งเข้าฟังก์ชัน
3. การใช้งานร่วมกัน (**Shared Reference**): เพื่อให้อยู่บนข้อมูลชุดเดียวกันจากหลายๆ จุดของโค้ด

การจัดการข้อผิดพลาดในภาษา Go (Error Handling in Go)

ในภาษา Go จะไม่มีระบบ **Exception (try-catch-finally)** เหมือนภาษาโปรแกรมอื่น แต่ Go ใช้วิธี **Explicit Error Handling** โดยฟังก์ชันที่อาจเกิดข้อผิดพลาดจะส่งคืนค่าผลลัพธ์คู่กับอินเทอร์เฟซ `error` เป็นค่าสุดท้ายเสมอ หากไม่มีข้อผิดพลาดเกิดขึ้นจะส่งคืนค่า `nil`

1. แนวคิดพื้นฐาน (Basic Concepts)

1.1 `error` Interface ใน Go

`error` ในภาษา Go เป็น Built-in Interface ที่มีข้อกำหนดง่ายๆ เพียงเมธอดเดียวดังนี้:

```
type error interface {  
    Error() string  
}
```

1.2 การตรวจสอบ Error (if err != nil)

รูปแบบมาตรฐาน (Idiomatic Go) ในการจัดการ Error คือการตรวจสอบว่า `err != nil` หรือไม่ทันทีหลังเรียกใช้งานฟังก์ชัน:

```
result, err := divide(10, 2)  
if err != nil {  
    // จัดการข้อผิดพลาดเมื่อเกิด error  
    fmt.Println("Error:", err)  
    return  
}  
// ทำงานต่อหากไม่มี error  
fmt.Println("Result:", result)
```

2. การสร้าง Error ขึ้นใช้งาน (Creating Errors)

2.1 การใช้ `errors.New()`

เหมาะสำหรับสร้างข้อผิดพลาดที่มีข้อความคงที่ สั้นกระชับ:

```
import "errors"

var ErrDivideByZero = errors.New("cannot divide by zero")
```

2.2 การใช้ `fmt.Errorf()`

เหมาะสำหรับสร้างข้อผิดพลาดที่ต้องการใส่ตัวแปรหรือฟอร์แมตข้อความแบบไดนามิก:

```
import "fmt"

func validateAge(age int) error {
    if age < 0 {
        return fmt.Errorf("invalid age %d: age cannot be negative", age)
    }
    return nil
}
```

2.3 การสร้าง Custom Error ชนิด Struct

เมื่อต้องการเก็บบริบทหรือข้อมูลของ Error เพิ่มเติม (เช่น Username, Error Code, Timestamp) เราสามารถสร้าง Struct และเขียนเมธอด `Error() string` เพื่อทำตามสัญญาของ `error` interface ได้:

```
type LoginError struct {
    Username string
    Message  string
}

// เมธอดนี้ทำให้ *LoginError ทำหน้าที่เป็น error interface
func (e *LoginError) Error() string {
    return fmt.Sprintf("Login error from username %s : %s", e.Username,
e.Message)
}
```

3. ตัวอย่างโค้ดประกอบการใช้งาน (Code Examples)

3.1 ตัวอย่างที่ 1: การส่งคืน Error พื้นฐานด้วย `errors.New()`

```
package main

import (
    "errors"
    "fmt"
)

// ฟังก์ชันหารตัวเลข ส่งคืนผลลัพธ์ (float64) และ error
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("cannot divide by zero") // ส่งคืน error เมื่อตัวหารเป็น 0
    }
    return a / b, nil // ส่งคืน nil เมื่อคำนวณสำเร็จไม่มีข้อผิดพลาด
}

func main() {
    res, err := divide(10, 0)
    if err != nil {
        fmt.Println("Error:", err) // Output: Error: cannot divide by zero
    } else {
        fmt.Println("Result:", res)
    }
}
```

3.2 ตัวอย่างที่ 2: การใช้ Custom Error Struct (LoginError)

```
package main

import "fmt"

// 1. ประกาศ Custom Struct สำหรับเก็บรายละเอียด Error
type LoginError struct {
    Username string
    Message  string
}

// 2. สร้างเมธอด Error() string เพื่อให้ LoginError ทำหน้าที่เป็น error interface
func (e *LoginError) Error() string {
    return fmt.Sprintf("Login error from username %s : %s", e.Username,
e.Message)
}

// 3. ฟังก์ชันตรวจสอบการเข้าสู่ระบบ
func login(username, password string) error {
    if username != "admin" || password != "123456" {
        return &LoginError{
            Username: username,
            Message:  "Invalid credentials",
        }
    }
    return nil // คืนค่า nil แสดงว่าเข้าสู่ระบบสำเร็จ
}

func main() {
    // ทดลองเรียกใช้ด้วยข้อมูลที่ไม่ถูกต้อง
    err := login("testuser", "56789")
    if err != nil {
        fmt.Println(err)
        // Output: Login error from username testuser : Invalid credentials
    } else {
        fmt.Println("Login successful")
    }
}
```

4. เทคนิคและแนวปฏิบัติที่ดี (Best Practices)

หัวข้อ	คำอธิบาย
Return Error Last	กำหนดให้ค่า <code>error</code> เป็น Return Value ตัวสุดท้ายของฟังก์ชันเสมอ เช่น <code>(int, error)</code>
Check Immediately	ตรวจสอบ <code>if err != nil</code> ทันทีหลังเรียกฟังก์ชันที่อาจเกิดข้อผิดพลาด
Early Return (Guard Clause)	หากเกิด Error ให้จัดการแล้ว <code>return</code> ทันที เพื่อหลีกเลี่ยงการเขียน <code>if-else</code> ซ้อนกันหลายชั้น
Custom Error Types	สร้าง Struct ทำตาม <code>error</code> interface เพื่อเก็บข้อมูลบริบทของ Error เพิ่มเติม
Panic & Recover	ใช้เฉพาะกรณีวิกฤตที่โปรแกรมไม่สามารถรันต่อได้จริงๆ ไม่ควรใช้ <code>panic</code> แทนการคืนค่า <code>error</code> ทั่วไป

[กลับไปสารบัญ](#)

การใช้งาน HTTP Server และ Client ในภาษา Go (HTTP in Go)

ภาษา Go มี Standard Library ที่ชื่อว่า `net/http` ซึ่งมีความสามารถสูงและประสิทธิภาพดีเยี่ยม สามารถใช้สร้างทั้ง HTTP Web Server และ HTTP Client ได้โดยไม่ต้องพึ่งพา Third-party Library เพิ่มเติม

1. การสร้าง HTTP Server (HTTP Web Server)

1.1 การสร้าง Server เบื้องต้นด้วย `http.HandleFunc`

เราสามารถลงทะเบียน Route และ Handler Function ได้ง่ายๆ ผ่าน `http.HandleFunc` และสั่งให้ Server เริ่มทำงานด้วย `http.ListenAndServe` :

```
package main

import (
    "fmt"
    "net/http"
)

func helloHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello, World!")
}

func main() {
    // กำหนด Handler สำหรับ Route "/"
    http.HandleFunc("/", helloHandler)

    fmt.Println("Server running on http://localhost:8080")
    // เริ่ม Server ที่ Port 8080
    if err := http.ListenAndServe(":8080", nil); err != nil {
        fmt.Println("Server error:", err)
    }
}
```

2. การจัดการ Request และ Response (Handling Requests & Responses)

2.1 การรับ-ส่งข้อมูลแบบ JSON

การสร้าง REST API ใน Go นิยมใช้ `json.NewDecoder` สำหรับอ่าน Request Body และ `json.NewEncoder` สำหรับส่ง JSON Response

```
package main

import (
    "encoding/json"
    "net/http"
)

type User struct {
    ID    int    `json:"id"`
    Name string `json:"name"`
}

func userHandler(w http.ResponseWriter, r *http.Request) {
    // กำหนด Header ให้เป็น application/json
    w.Header().Set("Content-Type", "application/json")

    if r.Method == http.MethodPost {
        var u User
        // แกะข้อมูล JSON จาก Request Body
        err := json.NewDecoder(r.Body).Decode(&u)
        if err != nil {
            http.Error(w, err.Error(), http.StatusBadRequest)
            return
        }

        // ส่ง Response กลับ
        w.WriteHeader(http.StatusCreated)
        json.NewEncoder(w).Encode(u)
        return
    }

    // กรณี GET Method
    u := User{ID: 1, Name: "Mikelopster"}
    json.NewEncoder(w).Encode(u)
}
```

2.2 การอ่าน Query Parameters

```
func searchHandler(w http.ResponseWriter, r *http.Request) {
    query := r.URL.Query().Get("q") // ดึงค่าจาก ?q=...
    fmt.Fprintf(w, "Search query: %s", query)
}
```

3. การใช้งาน HTTP Client (Sending HTTP Requests)

3.1 การส่ง HTTP GET Request

```
package main

import (
    "fmt"
    "io"
    "net/http"
)

func main() {
    resp, err := http.Get("https://api.github.com/users/octocat")
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    defer resp.Body.Close() // ต้องปิด Body หลังใช้งานเสร็จเสมอ

    body, err := io.ReadAll(resp.Body)
    if err != nil {
        fmt.Println("Error reading body:", err)
        return
    }

    fmt.Println("Response Status:", resp.Status)
    fmt.Println("Response Body:", string(body))
}
```

3.2 การสร้าง HTTP Client พร้อมกำหนด Timeout

เพื่อป้องกันไม่ให้ Client รอนานเกินไป ควรสร้าง `http.Client` และกำหนด Timeout:


```
import (  
    "net/http"  
    "time"  
)  
  
client := &http.Client{  
    Timeout: 10 * time.Second,  
}  
  
resp, err := client.Get("https://example.com")
```

4. สรุปแนวปฏิบัติที่ดี (Best Practices)

- ปิด Response Body** เสมอ: เมื่อใช้งาน HTTP Client ต้องใช้ `defer resp.Body.Close()` เพื่อป้องกัน Resource Leak
- ตั้งค่า Timeout**: ควรตั้งค่า Timeout สำหรับทั้ง HTTP Server และ Client เสมอเพื่อความปลอดภัย
- ตรวจสอบ HTTP Status Code**: ควรตรวจสอบ `resp.StatusCode` หรือกำหนด `w.WriteHeader()` ให้ถูกต้องตามมาตรฐาน HTTP REST API

[กลับไปสารบัญ](#)

การใช้งาน Go Fiber Framework (Fiber in Go)

Fiber เป็น Web Framework สำหรับภาษา Go ที่ถูกออกแบบมาโดยได้รับแรงบันดาลใจจาก **Express.js** (Node.js) ทำงานอยู่บน **Fasthttp** (Engine HTTP ที่เร็วที่สุดใน Go) ทำให้ประมวลผลได้รวดเร็วและใช้หน่วยความจำต่ำมาก เหมาะสำหรับการพัฒนา RESTful API และ Microservices

1. การติดตั้งและการใช้งานเบื้องต้น (Installation & Basic Setup)

1.1 การติดตั้ง Fiber v2

สามารถติดตั้งแพ็คเกจผ่าน `go get` :

```
go get -u github.com/gofiber/fiber/v2
```

1.2 ตัวอย่าง Server เบื้องต้น

```
package main

import (
    "log"

    "github.com/gofiber/fiber/v2"
)

func main() {
    // สร้าง Fiber App instance
    app := fiber.New()

    // กำหนด Route GET "/"
    app.Get("/", func(c *fiber.Ctx) error {
        return c.SendString("Hello, World 🙋!")
    })

    // เริ่มต้น Server ที่ Port 3000
    log.Fatal(app.Listen(":3000"))
}
```

2. การจัดการ Routing และ Parameters (Routing & Parameters)

2.1 HTTP Methods และ Path Parameters

ใช้ `c.Params()` เพื่อดึงค่าจาก URL Path parameter:

```
// Path Parameter: GET /users/123
app.Get("/users/:id", func(c *fiber.Ctx) error {
    id := c.Params("id")
    return c.SendString("User ID: " + id)
})

// Optional Parameter: GET /hello หรือ GET /hello/john
app.Get("/hello/:name?", func(c *fiber.Ctx) error {
    name := c.Params("name", "Guest") // ถ้าไม่มีค่า ให้ใส่ default เป็น Guest
    return c.SendString("Hello " + name)
})
```

2.2 Query Parameters

ใช้ `c.Query()` เพื่ออ่านค่า Query String เช่น `/search?q=golang` :

```
app.Get("/search", func(c *fiber.Ctx) error {
    searchTerm := c.Query("q", "default-term")
    return c.JSON(fiber.Map{
        "query": searchTerm,
    })
})
```

3. การรับส่งข้อมูลด้วย JSON (Handling JSON Request & Response)

3.1 การส่ง JSON Response

ใช้ `c.JSON()` หรือ `fiber.Map{}` สำหรับส่งคำตอบกลับในรูปแบบ JSON:

```

type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name"`
}

app.Get("/api/user", func(c *fiber.Ctx) error {
    user := User{ID: 1, Name: "Mikelopster"}
    return c.Status(fiber.StatusOK).JSON(user)
})

```

3.2 การรับข้อมูลด้วย BodyParser

ใช้ `c.BodyParser()` เพื่อแปลงข้อมูล JSON ใน Request Body เข้าสู่ Struct:

```

app.Post("/api/users", func(c *fiber.Ctx) error {
    user := new(User)

    // Bind JSON Body เข้าสู่ Struct
    if err := c.BodyParser(user); err != nil {
        return c.Status(fiber.StatusBadRequest).JSON(fiber.Map{
            "error": "Cannot parse JSON",
        })
    }

    return c.Status(fiber.StatusCreated).JSON(user)
})

```

4. การจัดการ Middleware และ Grouping (Middleware & Route Grouping)

4.1 Built-in Middleware (Logger, CORS, Recover)

Fiber มี Middleware ขอดนิยมเตรียมไว้ให้ในตัว:

```
import (  
    "github.com/gofiber/fiber/v2/middleware/cors"  
    "github.com/gofiber/fiber/v2/middleware/logger"  
    "github.com/gofiber/fiber/v2/middleware/recover"  
)  
  
app.Use(logger.New()) // บันทึก Request Log  
app.Use(recover.New()) // ป้องกัน Server ล่มจาก Panic  
app.Use(cors.New()) // เปิดใช้งาน CORS
```

4.2 Route Grouping (จัดกลุ่ม API)

จัดหมวดหมู่ Endpoint เช่น `/api/v1` :

```
api := app.Group("/api/v1")  
  
api.Get("/products", getProducts)  
api.Post("/products", createProduct)
```

5. สรุปเปรียบเทียบ net/http vs Fiber

คุณสมบัติ	net/http (Standard Library)	Fiber Framework
ความสะดวกในการ Routing	ต้องจัดการ Pattern เอง	มี Router สำเร็จรูปสไตล์ Express.js
JSON Response	ต้องใช้โครงสร้างประกอบ <code>json.Encoder</code>	มี <code>c.JSON()</code> และ <code>c.BodyParser()</code>
Middleware	เขียนแบบ Wrapper Pattern เอง	มี Middleware ขนัดพร้อมใช้งานหลากหลาย
Performance	สูงมาก (Standard Go)	สูงมากเป็นพิเศษ (ขับเคลื่อนด้วย Fasthttp)

[กลับไปสารบัญ](#)

การสร้าง RESTful API ด้วย Fiber (Fiber CRUD API)

การพัฒนา **RESTful API** ด้วย **Fiber Framework** ร่วมกับแนวทางการออกแบบโครงสร้างโปรเจกต์ที่เป็นสัดส่วนตามมาตรฐานภาษา Go (Package-based Project Structure) และการจัดการหน่วยความจำแบบพึ่งพาตัวเองได้ (In-memory Storage) ที่รองรับ **Concurrency (Thread-safety)**

1. โครงสร้างสถาปัตยกรรมโปรเจกต์ (Modular Project Structure)

การปรับปรุงโครงสร้างโปรเจกต์จากไฟล์เดี่ยวไปเป็นแบบแยก **Packages** ช่วยให้โค้ดทดสอบง่ายขึ้น (Testable), บำรุงรักษาง่าย (Maintainable) และขยายขนาดได้ดี (Scalable):

```
2-fiber/
├── main.go                # Entry Point ของแอปพลิเคชัน (ตั้งค่า Server &
Routes)
├── model/
│   └── book.go            # Domain Model (package model)
├── request/
│   └── book_request.go    # Request DTOs (package request)
├── store/
│   ├── book_store.go     # In-memory Store & Mutex (package store)
│   └── book_store_test.go # Unit Test สำหรับ Store
└── handler/
    ├── book_handler.go   # HTTP Handlers / Controllers (package handler)
    └── book_handler_test.go # Unit Test สำหรับ Handlers
```

2. โครงสร้างข้อมูลและ DTOs (Models & Requests)

2.1 Domain Model (model/book.go)

นิยาม Struct สำหรับข้อมูลหนังสือภายในระบบ:

```
package model

type Book struct {
    ID      int    `json:"id"`
    Title   string `json:"title"`
    Author  string `json:"author"`
}
```

2.2 Request DTO (request/book_request.go)

แยก Struct สำหรับรับ Request Payload จาก Client เพื่อป้องกันไม่ให้ภายนอกระบุ ID เข้ามาเอง:

```
package request

type BookRequest struct {
    Title string `json:"title"`
    Author string `json:"author"`
}
```

3. In-Memory Data Store แบบ Thread-Safe (store/book_store.go)

การจัดการข้อมูลในหน่วยความจำโดยใช้ `sync.RWMutex` เพื่อป้องกัน **Data Race** เมื่อมีหลาย Goroutines เข้าถึงพร้อมกัน:

```
package store

import (
    "sync"
    "github.com/oopbest/go-fiber/model"
)

type BookStore struct {
    mu      sync.RWMutex
    books   []model.Book
    nextID  int
}
```

3.1 การดึงข้อมูลแบบอ่านเท่านั้น (Read Lock & Defensive Copy)

```
func (s *BookStore) All() []model.Book {
    s.mu.RLock()
    defer s.mu.RUnlock()

    // คัดลอก (Copy) Slice ใหม่ ป้องกัน Data Race เมื่อถูกนำไปใช้นอก Mutex
    return append([]model.Book(nil), s.books...)
}
```

3.2 การเพิ่มและแก้ไขข้อมูล (Write Lock)

```
func (s *BookStore) Create(title, author string) model.Book {
    s.mu.Lock()
    defer s.mu.Unlock()

    book := model.Book{
        ID:      s.nextID,
        Title:   title,
        Author:  author,
    }
    s.nextID++
    s.books = append(s.books, book)
    return book
}
```


3.3 เทคนิคการลบข้อมูลออกจาก Slice (Slice Deletion Trick)

```
func (s *BookStore) Delete(id int) bool {
    s.mu.Lock()
    defer s.mu.Unlock()

    for i, book := range s.books {
        if book.ID == id {
            // ดัดสมาชิกตำแหน่ง i ออก โดยนำส่วนหน้า (0 ถึง i-1) มาต่อกับส่วนหลัง (i+1 ถึง
จบ)
            s.books = append(s.books[:i], s.books[i+1:]...)
            return true
        }
    }
    return false
}
```

4. HTTP Handlers & Controllers (handler/book_handler.go)

ควบคุมการทำงานผ่าน HTTP Context ของ Fiber โดยรับส่งข้อมูลร่วมกับ BookStore :

```
package handler

type BookHandler struct {
    store *store.BookStore
}

func NewBookHandler(store *store.BookStore) *BookHandler {
    return &BookHandler{store: store}
}
```

Endpoints Table

HTTP Method	Path	Handler Method	Status Code	คำอธิบาย
GET	/books	GetAllBooks	200 OK	ดึงข้อมูลหนังสือทั้งหมด
GET	/books/:id	GetBookByID	200 OK	ดึงข้อมูลหนังสือตาม ID
POST	/books	CreateBook	201 Created	สร้างหนังสือเล่มใหม่
PUT	/books/:id	UpdateBook	200 OK	แก้ไขข้อมูลหนังสือตาม ID

HTTP Method	Path	Handler Method	Status Code	คำอธิบาย
DELETE	/books/:id	DeleteBook	200 OK	ลบหนังสือตาม ID

5. Main Entry Point (main.go)

ไฟล์ `main.go` ทำหน้าที่เพียงแต่การประกอบ dependencies (Dependency Injection) และตั้งค่า HTTP Router:

```
package main

import (
    "log"
    "github.com/gofiber/fiber/v2"
    "github.com/oopbest/go-fiber/handler"
    "github.com/oopbest/go-fiber/model"
    "github.com/oopbest/go-fiber/store"
)

func main() {
    bookStore := store.NewBookStore([]model.Book{
        {ID: 1, Title: "Book 1", Author: "Author 1"},
    })
    bookHandler := handler.NewBookHandler(bookStore)

    app := fiber.New()
    app.Get("/books", bookHandler.GetAllBooks)
    app.Get("/books/:id", bookHandler.GetBookByID)
    app.Post("/books", bookHandler.CreateBook)
    app.Put("/books/:id", bookHandler.UpdateBook)
    app.Delete("/books/:id", bookHandler.DeleteBook)

    log.Fatal(app.Listen(":8080"))
}
```

6. ข้อดีของการแยก Package และทำ Unit Testing

- Separation of Concerns:** แต่ละแพ็คเกจรับผิดชอบหน้าที่ของตนเองชัดเจน (`model` , `request` , `store` , `handler`)
- Unit Testing:** สามารถเขียนทดสอบ `book_store_test.go` และ `book_handler_test.go` แยกชิ้นส่วนได้อย่างอิสระและครอบคลุม
- Clean Main File:** `main.go` เหลือขนาดสั้น สอาด และอ่านเข้าใจง่าย

การจัดการ Environment Variables ในภาษา Go (Environment Variables in Go)

Environment Variables (ตัวแปรสภาพแวดล้อม) เป็นกลไกสำคัญในการแยกการตั้งค่าของแอปพลิเคชัน (Configuration) ออกจากซอร์สโค้ดตามหลักการ **12-Factor App** เพื่อความปลอดภัยในการเก็บข้อมูลลับ (Secrets, API Keys, Database Password) และรองรับการทำงานหลายสภาพแวดล้อม (Development, Staging, Production)

1. การอ่านค่าด้วย Standard Library (os Package)

1.1 การใช้ os.Getenv()

ดึงค่าตัวแปรสภาพแวดล้อมตาม Key ที่กำหนด หากไม่มีการตั้งค่าไว้จะคืนค่าเป็น String ว่าง (""):

```
package main

import (
    "fmt"
    "os"
)

func main() {
    port := os.Getenv("PORT")
    if port == "" {
        port = "8080" // ค่า Default กรณีไม่ได้กำหนดไว้
    }
    fmt.Println("Server running on port:", port)
}
```

1.2 การใช้ os.LookupEnv()

ใช้เมื่อต้องการแยกแยะระหว่าง "ตัวแปรที่ไม่ได้ถูกตั้งค่า" กับ "ตัวแปรที่ตั้งค่าเป็น String ว่าง":

```
val, exists := os.LookupEnv("SECRET_KEY")
if !exists {
    fmt.Println("ไม่มีการตั้งค่า SECRET_KEY ในระบบ")
}
```

1.3 การเขียน Helper Function ยุบรวมรองรับทั้ง บังคับมีค่า (Require) และ ค่าเริ่มต้น (Default)

สามารถใช้ **Variadic Parameters** (`...string`) เพื่อสร้างฟังก์ชันอ่าน Env แบบอเนกประสงค์:

```
func getEnv(key string, defaultValue ...string) string {
    value := os.Getenv(key)
    if value != "" {
        return value
    }
    if len(defaultValue) > 0 {
        return defaultValue[0]
    }
    log.Fatalf("Environment variable %s is required but not set", key)
    return ""
}

// วิธีใช้งาน:
adminPass := getEnv("ADMIN_PASSWORD")           // บังคับต้องมี ถ้าไม่มีจะ log.Fatal
ทันที
jwtSecret := getEnv("JWT_SECRET", "default_secret") // ถ้าไม่มีจะใช้
"default_secret" แทน
```

2. การใช้งานไฟล์ `.env` ด้วยแพ็คเกจ `godotenv`

ในขั้นตอนการพัฒนา (Development) นิยมเก็บค่าตัวแปรไว้ในไฟล์ `.env` และใช้แพ็คเกจ github.com/joho/godotenv ในการโหลดเข้าสู่โปรเซสโดยอัตโนมัติ

2.1 การติดตั้ง

```
go get github.com/joho/godotenv
```

2.2 ตัวอย่างไฟล์ `.env`

```
PORT=8080
SECRET=my_super_secret_key_123
DB_URL=postgres://user:pass@localhost:5432/mydb
```

2.3 การโหลดเข้าใช้งานใน `main.go`

```
package main

import (
    "log"
    "os"

    "github.com/joho/godotenv"
)

func main() {
    // โหลดไฟล์ .env เข้าสู่ os.Getenv
    if err := godotenv.Load(); err != nil {
        log.Println("Warning: No .env file found")
    }

    secret := os.Getenv("SECRET")
    log.Printf("Loaded SECRET: %s", secret)
}
```

ข้อควรระวัง ห้าม **commit** ไฟล์ `.env` เข้า **Git Repository** โดยเด็ดขาด! ควรเพิ่ม `.env` ไว้ในไฟล์ `.gitignore` เสมอ และสร้างไฟล์ตัวอย่างชื่อ `.env.example` ไว้เป็นเทมเพลตแทน

3. การกำหนด Environment Variables ผ่าน CLI สั้น

สามารถส่งค่าตัวแปรสภาพแวดล้อมได้ผ่านคำสั่ง Terminal ในแต่ละระบบปฏิบัติการ:

Git Bash / Linux / macOS

```
SECRET=mysecretkey go run main.go
# หรือใช้กับ air (Live Reload)
SECRET=mysecretkey air
```

PowerShell (Windows)

```
$env:SECRET="mysecretkey"; go run main.go
```

Command Prompt (CMD Windows)

```
set SECRET=mysecretkey && go run main.go
```

4. สรุปแนวปฏิบัติที่ดี (Best Practices)

1. ไม่ **Hardcode** ข้อมูลลับ: ห้ามเก็บ Secret, Passwords หรือ API Keys ไว้ในโค้ด Go
2. ใช้ **.env** เฉพาะการพัฒนาในเครื่อง: ในระบบ Production ควรส่งค่าผ่าน Docker, Kubernetes Secrets หรือ Cloud Provider (AWS Secrets Manager)
3. ใช้ **.env.example**: สร้างไฟล์ **.env.example** ระบุเฉพาะชื่อ Key โดยไม่ใส่ค่าจริง เพื่อให้คนใหม่รู้ว่าต้องตั้งค่าตัวแปรใดบ้าง

[กลับไปสารบัญ](#)

การใช้งาน Middleware ในภาษา Go และ Fiber (Middleware in Go & Fiber)

Middleware คือ ฟังก์ชันที่ทำหน้าที่เป็น "ตัวกลาง" แทรกอยู่ระหว่างที่ **HTTP Request** เข้ามา จนกระทั่งส่ง **HTTP Response** กลับออกไป ใน Request-Response Pipeline ใช้สำหรับประมวลผลงานส่วนกลางที่ทุก Endpoint ต้องใช้ร่วมกัน เช่น การบันทึก Log, การทำ Authentication, การดักจับ Panic หรือการกำหนด CORS

1. ลำดับการทำงานของ Middleware (Middleware Pipeline)



2. Built-in Middlewares ใน Fiber Framework

Fiber มี Middleware ยอดนิยมเตรียมมาให้พร้อมใช้งานในตัว สามารถเรียกใช้ผ่าน `app.Use()` :

2.1 Logger Middleware

เก็บบันทึกข้อมูล HTTP Request แต่ละรายการ (Method, Path, Status Code, Latency):


```
import "github.com/gofiber/fiber/v2/middleware/logger"

app := fiber.New()
app.Use(logger.New()) // บันทึก Log ทุกครั้งที่มี Request เข้ามา
```

2.2 Recover Middleware

ดักจับข้อผิดพลาดรุนแรงระดับ **Panic** ไม่ให้ส่งผลให้ Web Server ล่ม (Crash) และเปลี่ยนให้ส่ง HTTP Status **500 Internal Server Error** แทน:

```
import "github.com/gofiber/fiber/v2/middleware/recover"

app.Use(recover.New())
```

2.3 CORS Middleware

จัดการสิทธิ์การเข้าถึง API ข้ามโดเมน (Cross-Origin Resource Sharing):

```
import "github.com/gofiber/fiber/v2/middleware/cors"

app.Use(cors.New(cors.Config{
    AllowOrigins: "https://example.com, https://myfrontend.com",
    AllowHeaders: "Origin, Content-Type, Accept, Authorization",
}))
```

3. การเขียน Custom Middleware ขึ้นใช้งานเอง (Custom Middleware)

ฟังก์ชัน Middleware ใน Fiber จะรับพารามิเตอร์ `c *fiber.Ctx` และต้องคืนค่าเป็น `error` หากต้องการส่งคำขอไปยัง Handler ถัดไป ต้องเรียก `c.Next()`

3.1 ตัวอย่าง: Secret API Key Check Middleware

สร้าง Middleware สำหรับตรวจสอบว่า Request ที่ส่งมามี `X-Secret-Key` ตรงตามที่กำหนดหรือไม่:

```

package main

import (
    "os"
    "github.com/gofiber/fiber/v2"
)

func SecretAuthMiddleware() fiber.Handler {
    return func(c *fiber.Ctx) error {
        secretKey := c.Get("X-Secret-Key")
        expectedSecret := os.Getenv("SECRET")

        if secretKey == "" || secretKey != expectedSecret {
            return c.Status(fiber.StatusUnauthorized).JSON(fiber.Map{
                "error": "Unauthorized: Invalid or missing Secret Key",
            })
        }

        // ผ่านการตรวจสอบ ส่งต่อไปยัง Handler ถัดไป
        return c.Next()
    }
}

```

3.2 ตัวอย่าง: Request Duration / Timing Middleware

คำนวณเวลาที่ใช้ในการประมวลผล Request แต่ละครั้ง:

```

func RequestTimer() fiber.Handler {
    return func(c *fiber.Ctx) error {
        start := time.Now()

        // ส่งให้ Handler ประมวลผลก่อน
        err := c.Next()

        duration := time.Since(start)
        log.Printf("[%s] %s took %v", c.Method(), c.Path(), duration)

        return err
    }
}

```

3.3 ตัวอย่าง: Role-Based Access Control (RBAC) Middleware

ตรวจสอบว่าผู้ใช้งานที่ยิง Request เข้ามามีสิทธิ์ (Role) ตรงตามที่กำหนดหรือไม่ (เช่น ต้องเป็น "admin" เท่านั้น):

```

func RequireRole(requiredRole string) fiber.Handler {
    return func(c *fiber.Ctx) error {
        userLocals := c.Locals("user")
        if userLocals == nil {
            return c.Status(fiber.StatusUnauthorized).JSON(fiber.Map{
                "error": "Authentication required",
            })
        }

        claims, ok := userLocals.(jwt.MapClaims)
        if !ok || claims["role"] != requiredRole {
            return c.Status(fiber.StatusForbidden).JSON(fiber.Map{
                "error": "Access denied: insufficient permissions",
            })
        }

        return c.Next()
    }
}

```

4. การประยุกต์ใช้งานร่วมกับ Route Grouping

สามารถเลือกให้ Middleware ทำงานเฉพาะกับกลุ่ม API ที่กำหนดได้โดยใช้ `app.Group()` :

```

app := fiber.New()

// Public Endpoints (ไม่ต้องยืนยันตัวตน)
app.Get("/public", func(c *fiber.Ctx) error {
    return c.SendString("Public Content")
})

// Protected Endpoints Group (ใช้ SecretAuthMiddleware)
api := app.Group("/api/v1", SecretAuthMiddleware())

api.Get("/books", bookHandler.GetAllBooks)
api.Post("/books", bookHandler.CreateBook)

```

5. สรุปประโยชน์ของ Middleware

1. **Reusability:** เขียน Logic สำหรับตรวจสอบหรือบันทึกข้อมูลเพียงครั้งเดียว แต่ใช้ร่วมกันได้ทุก Endpoint

2. **Clean Code (Separation of Concerns):** แยกโค้ดตรวจสอบสิทธิ์ (Auth/Validation) ออกจาก Business Logic ของ Controller/Handler
3. **Security & Reliability:** ช่วยเพิ่มความปลอดภัย (CORS, Auth) และความเสถียร (Recover จาก Panic) ให้กับระบบ API

[กลับไปสารบัญ](#)

การสร้างเอกสาร API ด้วย Swagger (Swagger Documentation in Go & Fiber)

Swagger (OpenAPI 3.0) เป็นมาตรฐานการอธิบายสถาปัตยกรรม RESTful API ที่ช่วยให้ผู้พัฒนาสามารถอ่านเอกสารและทดสอบ API ผ่านหน้าเว็บเบราว์เซอร์ (Interactive UI) ได้โดยไม่ต้องพึ่งพาคำอธิบายจากภายนอก

1. แพคเกจที่ต้องใช้งาน (Core Packages)

- `github.com/gofiber/swagger` : Fiber Middleware สำหรับให้บริการหน้าเว็บ Swagger UI ผ่าน HTTP Endpoint
- `github.com/swaggo/swag` : เครื่องมือแปลง Annotations ที่เขียนกำกับฟังก์ชัน Go ให้กลายเป็นไฟล์ OpenAPI Specs (`docs/docs.go` , `swagger.json` , `swagger.yaml`)

การติดตั้ง

```
go get github.com/gofiber/swagger
go get github.com/swaggo/swag
```

2. การระบุ General API Annotations ใน `main.go`

กำหนดข้อมูลพื้นฐานของ API และระบบความปลอดภัย (Security Scheme) ไว้เหนือฟังก์ชัน `main()` ใน `main.go`:

```

package main

import (
    "github.com/gofiber/swagger"
    _ "github.com/oopbest/go-fiber/docs" // นำเข้าแพ็คเกจ docs ที่ถูก swag init เจน
    ออกมา
)

// @title           Bookstore Fiber API
// @version          1.0
// @description      RESTful API for Bookstore Management built with Fiber, JWT
// Auth, and RBAC.
// @host             localhost:8080
// @BasePath         /

// @securityDefinitions.apikey BearerAuth
// @in header
// @name Authorization
// @description Type 'Bearer ' followed by your JWT token. Example: "Bearer
// eyJhbGci..."

func main() {
    app := fiber.New()

    // ลงทะเบียน Endpoint สำหรับเปิดหน้าเว็บ Swagger UI
    app.Get("/swagger/*", swagger.HandlerDefault)

    // ... (routes อื่นๆ) ...
}

```

3. การเขียน Swag Annotations กำกับใน Handlers

ใส่ Annotations เหนือแต่ละ Handler Function ในแพ็คเกจ handler :

3.1 ตัวอย่าง: Login Endpoint (POST /login)

```
// Login handles user authentication and generates a JWT token.
// @Summary      User Login
// @Description   Authenticate with username and password to get a JWT token.
// @Tags         Auth
// @Accept       json
// @Produce      json
// @Param        request body request.LoginRequest true "Login Credentials"
// @Success      200  {object}  map[string]interface{}
// @Failure      400  {object}  map[string]interface{}
// @Failure      401  {object}  map[string]interface{}
// @Router       /login [post]
func (h *AuthHandler) Login(c *fiber.Ctx) error { ... }
```

3.2 ตัวอย่าง: Protected Endpoint (POST /books)

ใช้ `@Security BearerAuth` เพื่อบอกว่า Endpoint นี้ต้องใช้ JWT Token ในการยืนยันตัวตน:

```
// CreateBook adds a new book to the store.
// @Summary      Create a new book
// @Description   Create a book (Requires JWT token with Admin role).
// @Tags         Books
// @Security     BearerAuth
// @Accept       json
// @Produce      json
// @Param        request body request.BookRequest true "Book Data"
// @Success      201  {object}  model.Book
// @Failure      401  {object}  map[string]interface{}
// @Failure      403  {object}  map[string]interface{}
// @Router       /books [post]
func (h *BookHandler) CreateBook(c *fiber.Ctx) error { ... }
```

3.3 ตัวอย่าง: Upload Endpoint (POST /upload)

```
// UploadFile saves an uploaded file to disk.
// @Summary      Upload a document file
// @Tags         Upload
// @Security      BearerAuth
// @Accept       mpfd
// @Produce      json
// @Param         file formData file true "File to upload"
// @Success       201 {object} map[string]string
// @Router       /upload [post]
func UploadFile(c *fiber.Ctx) error { ... }
```

4. คำสั่งสร้างไฟล์เอกสาร (Generating OpenAPI Specs)

เมื่อมีการแก้ไขหรือเพิ่ม Annotations ใหม่ ให้รันคำสั่งเพื่อสร้าง/อัปเดตไฟล์ในโฟลเดอร์ docs/ :

```
go run github.com/swaggo/swag/cmd/swag init
```

ผลลัพธ์ที่จะได้ในไดเรกทอรีโปรเจกต์:

```
2-fiber/
├── docs/
│   ├── docs.go          # Go source code สำหรับ import ใน main.go
│   ├── swagger.json     # JSON Spec OpenAPI 3.0
│   └── swagger.yaml     # YAML Spec OpenAPI 3.0
```

5. การเปิดดูและทดสอบยิง API (Interactive UI)

1. รันเซิร์ฟเวอร์ (go run main.go หรือ air)
2. เปิดเบราว์เซอร์เข้าที่ URL:

```
http://localhost:8080/swagger/index.html
```

3. กดปุ่ม **Authorize** ใส่ Bearer <jwt_token> เพื่อทดสอบ Endpoints ที่มีการคุ้มครองสิทธิ์ได้ทันทีผ่านหน้าเว็บ

การใช้งาน Go database/sql & PostgreSQL (Go Database/SQL)

สรุปการใช้งานแพ็คเกจมาตรฐาน `database/sql` ร่วมกับ PostgreSQL Driver (github.com/lib/pq) การจัดโครงสร้างโค้ดด้วย Repository Pattern และเทคนิคการแก้ปัญหาข้อผิดพลาดยอดนิยมในการพัฒนาโปรเจกต์ Go + Database

1. การตั้งค่าและการเชื่อมต่อฐานข้อมูล (Database Connection)

การเชื่อมต่อ PostgreSQL ใน Go จะใช้แพ็คเกจ `database/sql` ร่วมกับ Driver github.com/lib/pq โดยทำการโหลดค่าคอนฟิกจากไฟล์ `.env` ผ่าน `godotenv`

```

package config

import (
    "database/sql"
    "fmt"
    "log"
    "os"

    "github.com/joho/godotenv"
    _ "github.com/lib/pq" // PostgreSQL Driver (Anonymous Import)
)

func ConnectDB() (*sql.DB, error) {
    _ = godotenv.Load()

    dsn := fmt.Sprintf("host=%s port=%s user=%s password=%s dbname=%s
sslmode=disable",
        os.Getenv("POSTGRES_HOST"),
        os.Getenv("POSTGRES_PORT"),
        os.Getenv("POSTGRES_USER"),
        os.Getenv("POSTGRES_PASSWORD"),
        os.Getenv("POSTGRES_DB"),
    )

    db, err := sql.Open("postgres", dsn)
    if err != nil {
        return nil, fmt.Errorf("failed to open database: %w", err)
    }

    // ทดสอบการเชื่อมต่อกับฐานข้อมูล
    if err := db.Ping(); err != nil {
        return nil, fmt.Errorf("failed to ping database: %w", err)
    }

    return db, nil
}

```

หมายเหตุ `_ "github.com/lib/pq"` เป็นการนำเข้าแพ็คเกจแบบ Blank Identifier เพื่อให้ฟังก์ชัน `init()` ของ Driver ทำการลงทะเบียน (Register) เข้ากับ `database/sql`

2. 🏗️ การสร้าง Schema และตารางข้อมูล (Table Initialization)

การสร้างตารางและกำหนดความสัมพันธ์ (Foreign Key Constraint) ด้วย `db.Exec()` :

```
func InitTables(db *sql.DB) error {
    supplierTableSQL := `
    CREATE TABLE IF NOT EXISTS suppliers (
        id SERIAL PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        phone VARCHAR(20),
        email VARCHAR(100),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );`

    productTableSQL := `
    CREATE TABLE IF NOT EXISTS products (
        id SERIAL PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        price NUMERIC(10,2) NOT NULL DEFAULT 0.00,
        stock INT NOT NULL DEFAULT 0,
        supplier_id INT REFERENCES suppliers(id) ON DELETE SET NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );`

    if _, err := db.Exec(supplierTableSQL); err != nil {
        return fmt.Errorf("failed to create suppliers table: %w", err)
    }
    if _, err := db.Exec(productTableSQL); err != nil {
        return fmt.Errorf("failed to create products table: %w", err)
    }
    return nil
}
```

3. 📁 การออกแบบโครงสร้างโปรเจกต์ (Repository Pattern & Constructor)

การแยกส่วนการทำงานออกเป็น Package ตามหลัก Clean Architecture เพื่อความง่ายในการดูแลรักษาและทดสอบ:

```
3-go-database/
├── config/
│   └── db.go          # จัดการ Connection และ Schema Initialization
├── models/
│   ├── supplier.go    # Struct ของ Supplier
│   └── product.go      # Struct ของ Product และ ProductWithSupplier
├── repository/
│   ├── supplier_repo.go # CRUD Operations สำหรับ Supplier
│   └── product_repo.go  # CRUD Operations สำหรับ Product
├── .env               # Environment Variables
└── main.go            # Entry Point สั่งรันโปรแกรม
```

การสร้าง Constructor และ Dependency Injection

การใช้ดีไซน์แบบ **Dependency Injection** ช่วยให้ทุก Repository สามารถใช้วัตถุ `*sql.DB` จาก `main.go` ร่วมกันได้:

```
type ProductRepository struct {
    DB *sql.DB
}

// Constructor Function สไตล์ Go (ขึ้นต้นด้วย New...)
func NewProductRepository(db *sql.DB) *ProductRepository {
    return &ProductRepository{DB: db}
}
```

4. 📊 เปรียบเทียบ 3 คำสั่งหลักในการรัน SQL ใน Go

คำสั่ง (Method)	การใช้งาน (Use Case)	คืนค่า (Return)	ตัวอย่างการนำไปใช้
<code>db.Exec()</code>	คำสั่งที่ไม่ต้องการผลลัพธ์ข้อมูล กลับมา	<code>sql.Result</code> , <code>error</code>	<code>INSERT</code> (ปกติ), <code>UPDATE</code> , <code>DELETE</code> , <code>CREATE TABLE</code>
<code>db.Query()</code>	คำสั่งที่ดึงข้อมูล หลายแถว (Multiple Rows)	<code>*sql.Rows</code> , <code>error</code>	<code>SELECT</code> รายการทั้งหมด (<code>for</code> <code>rows.Next()</code>)
<code>db.QueryRow()</code>	คำสั่งที่ดึงข้อมูล แถวเดียว (Single Row)	<code>*sql.Row</code>	<code>SELECT WHERE id = \$1</code> , <code>INSERT</code> ... <code>RETURNING id</code>

5. 📌 การทำ CRUD Operations (โค้ดตัวอย่าง)

1) CREATE & RETURNING ID (QueryRow)

เมื่อต้องการเพิ่มข้อมูลและรับค่า `id` ที่ถูกสร้างแบบ Auto-increment (SERIAL) กลับมา:

```
func (r *SupplierRepository) Create(s *models.Supplier) (int, error) {
    query := `INSERT INTO suppliers (name, phone, email) VALUES ($1, $2, $3)
RETURNING id, created_at`
    err := r.DB.QueryRow(query, s.Name, s.Phone, s.Email).Scan(&s.ID,
&s.CreatedAt)
    if err != nil {
        return 0, err
    }
    return s.ID, nil
}
```

2) READ ALL & JOIN (Query)

การอ่านข้อมูลหลายแถวร่วมกับการเชื่อมตาราง (LEFT JOIN) และการตรวจสอบ Data Stream Error:

```

func (r *ProductRepository) GetAll() ([]models.ProductWithSupplier, error) {
    query := `
        SELECT p.id, p.name, p.price, p.stock, p.supplier_id, p.created_at,
        s.name as supplier_name
        FROM products p
        LEFT JOIN suppliers s ON p.supplier_id = s.id
        ORDER BY p.id`

    rows, err := r.DB.Query(query)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var products []models.ProductWithSupplier
    for rows.Next() {
        var p models.ProductWithSupplier
        if err := rows.Scan(&p.ID, &p.Name, &p.Price, &p.Stock, &p.SupplierID,
        &p.CreatedAt, &p.SupplierName); err != nil {
            return nil, err
        }
        products = append(products, p)
    }

    // ⚠️ ตรวจสอบ error ที่อาจเกิดขึ้นระหว่างวนลูปอ่าน rows Stream
    if err := rows.Err(); err != nil {
        return nil, err
    }

    return products, nil
}

```

3) UPDATE & DELETE (Exec)

คำสั่งแก้ไขและลบข้อมูลใช้ `db.Exec()` และตรวจสอบผลลัพธ์การอัปเดตผ่าน `RowsAffected()` :

```

func (r *ProductRepository) Update(p *models.Product) error {
    query := `UPDATE products SET name = $1, price = $2, stock = $3, supplier_id
= $4 WHERE id = $5`
    res, err := r.DB.Exec(query, p.Name, p.Price, p.Stock, p.SupplierID, p.ID)
    if err != nil {
        return err
    }

    rowsAffected, err := res.RowsAffected()
    if err != nil {
        return err
    }
    if rowsAffected == 0 {
        return fmt.Errorf("product with id %d not found", p.ID)
    }
    return nil
}

```

6. 💡 เทคนิคการจัดการ Pointer และการพิมพ์ผลลัพธ์ (`fmt.Printf`)

ฟิลด์ที่ยอมให้เป็น NULL ในตาราง (เช่น Foreign Key `supplier_id`) ใน Go จะนิยมใช้ชนิดข้อมูลเป็น **Pointer** เช่น `*int` หรือ `*string`

การแสดงผลลัพธ์ที่ปลอดภัย (Safely Printing Pointers)

การนำ `*string` หรือ `*int` ไปใส่ใน `fmt.Printf` โดยตรงจะเกิด Warning หรือพิมพ์ Memory Address ออกมา จึงต้องตรวจสอบ `nil` และ Dereference ค่าเสมอ:

```

for _, p := range products {
    supID := 0
    if p.SupplierID != nil {
        supID = *p.SupplierID // Dereference ค่า int จาก Pointer
    }
    supName := "N/A"
    if p.SupplierName != nil {
        supName = *p.SupplierName // Dereference ค่า string จาก Pointer
    }

    fmt.Printf(" - ID: %d, Name: %s, Price: %.2f THB, Stock: %d, Supplier ID:
%d, Supplier Name: %s\n",
        p.ID, p.Name, p.Price, p.Stock, supID, supName)
}

```

7. 🛠️ สรุปข้อผิดพลาดพบบ่อย และแนวทางแก้ไข (Troubleshooting)

1. `sql.Rows "rows" is used in Next loop without final check of rows.Err()`

- สาเหตุ: `rows.Next()` จะคืนค่า `false` ทั้งเมื่ออ่านจบปกติ และเมื่อเกิด Network/Stream Error ระหว่างทาง
- แก้ไข: ต้องเขียน `if err := rows.Err(); err != nil` ปิดท้ายลูป `for rows.Next()` เสมอ

2. `pq: insert or update on table violates foreign key constraint (23503)`

- สาเหตุ: สั่งเพิ่มหรืออัปเดตข้อมูลโดยอ้างอิง Foreign Key ID (เช่น `supplier_id`) ที่ไม่มีอยู่จริงในตารางอ้างอิง
- แก้ไข: ตรวจสอบว่า ID ตารางแม่มีอยู่จริงใน Database ก่อนทำรายการ

3. `undefined: productId` หรือ `declared and not used: productId`

- สาเหตุ: ประกาศตัวแปรในหัวเงื่อนไข `if productId, err := ...; err != nil` ทำให้ตัวแปร `productId` มีตัวตนแค่ภายในบล็อก `if` เท่านั้น
- แก้ไข: ประกาศตัวแปรรับค่านอกบล็อก `if` ก่อนเรียกเช็ค `err`

4. `cannot use ... (value of type *models.ProductWithSupplier) as *models.Product value in assignment`

- สาเหตุ: Go เป็นภาษา Strictly Typed ไม่ยอมให้นำ Struct ต่างชนิดกันมา Re-assign ใส่ตัวแปรเดิม
- แก้ไข: ประกาศตัวแปรรับค่าใหม่ด้วยชื่อแยกต่างหาก เช่น `singleProduct`

8. ⚠️ ข้อควรจำสำคัญ (Best Practices)

สำคัญ

1. การปิด **Connection** (`defer rows.Close()`): ทุกครั้งที่ใช้ `db.Query()` ต้องสั่ง `defer rows.Close()` เพื่อคืน Resource ให้กับ Connection Pool
2. การเช็ค `rows.Err()` : ต้องตรวจสอบ `rows.Err()` หลังจบลูป `for rows.Next()` ทุกครั้ง
3. การจัดการค่า NULL : ใช้ Pointer (`*int` , `*string`) ใน Struct เพื่อรับค่า NULL ป้องกัน Scan error
4. **Parameterized Queries (\$1, \$2)**: ใช้การส่งค่าแบบ `$1` , `$2` เสมอ เพื่อป้องกันปัญหา **SQL Injection**

การจัดการ Database Transaction ใน Go (Database Transaction in Go)

สรุปการใช้งาน **Database Transaction** ในภาษา Go ด้วยแพ็คเกจมาตรฐาน `database/sql` เพื่อรักษารวมความถูกต้องของข้อมูลตามหลัก ACID การจัดการ Deadlock และเทคนิคระบบ Retry

1. แนวคิดหลักของ Database Transaction (ACID Properties)

Transaction คือการรวมชุดคำสั่ง SQL หลาย ๆ คำสั่งเข้าด้วยกันเป็น หน่วยการทำงานเดียว (Single Unit of Work) โดยปฏิบัติตามหลัก ACID:

- Atomicity (All or Nothing):** ต้องสำเร็จทั้งหมดทุกขั้นตอน หรือหากเกิดความผิดพลาดแม้แต่ขั้นตอนเดียว จะต้องยกเลิกคืนค่ากลับทั้งหมด (Rollback)
- Consistency:** ข้อมูลต้องถูกต้องตามกฎเกณฑ์และ Constraints ของฐานข้อมูลทั้งก่อนและหลังทำรายการ
- Isolation:** การทำงานของแต่ละ Transaction ที่เกิดขึ้นพร้อมกัน ต้องแยกออกจากกัน ไม่ส่งผลกระทบกัน
- Durability:** เมื่อทำการบันทึก (Commit) แล้ว ข้อมูลจะถูกบันทึกลงฐานข้อมูลอย่างถาวร

2. รูปแบบการใช้งานสไต์ล Go (Safe Transaction Pattern)

ใน Go จะเปิด Transaction ด้วย `db.Begin()` หรือ `db.BeginTx()` และใช้เทคนิค `defer tx.Rollback()` เพื่อความปลอดภัย:

```

func TransferMoney(db *sql.DB, fromAccID, toAccID int, amount float64) error {
    // 1. เริ่มต้น Transaction
    tx, err := db.Begin()
    if err != nil {
        return fmt.Errorf("failed to begin transaction: %w", err)
    }

    // 2. ป้องกัน Transaction ค้างด้วย defer tx.Rollback()
    // หากสิ่ง tx.Commit() สำเร็จแล้ว สิ่ง tx.Rollback() ใน defer จะไม่มีผลใดๆ (Safely
    ignored)
    defer tx.Rollback()

    // 3. ขั้นตอนที่ 1: ตัดเงินจากบัญชีผู้โอน (ใช้ tx.Exec ไม่ใช่ db.Exec)
    res1, err := tx.Exec("UPDATE accounts SET balance = balance - $1 WHERE id =
    $2 AND balance >= $1", amount, fromAccID)
    if err != nil {
        return fmt.Errorf("failed to deduct balance: %w", err)
    }
    rows1, _ := res1.RowsAffected()
    if rows1 == 0 {
        return fmt.Errorf("insufficient balance or account %d not found",
        fromAccID)
    }

    // 4. ขั้นตอนที่ 2: เพิ่มเงินเข้าบัญชีผู้รับ
    res2, err := tx.Exec("UPDATE accounts SET balance = balance + $1 WHERE id =
    $2", amount, toAccID)
    if err != nil {
        return fmt.Errorf("failed to add balance: %w", err)
    }
    rows2, _ := res2.RowsAffected()
    if rows2 == 0 {
        return fmt.Errorf("recipient account %d not found", toAccID)
    }

    // 5. หากทุกขั้นตอนสำเร็จ สิ่ง ยืนยันการทำรายการ (Commit)
    if err := tx.Commit(); err != nil {
        return fmt.Errorf("failed to commit transaction: %w", err)
    }

    return nil
}

```

3. 🛒 ตัวอย่างระบบจริง: การสร้าง Order และหัก Stock สินค้า

ตัวอย่างการบันทึกรายการคำสั่งซื้อ (Create Order) พร้อมหักจำนวนสินค้าคงคลัง (Deduct Stock):

```

type OrderReq struct {
    ProductID int `json:"product_id"`
    Quantity  int `json:"quantity"`
}

func CreateOrderTx(db *sql.DB, req OrderReq) (int, error) {
    tx, err := db.Begin()
    if err != nil {
        return 0, err
    }
    defer tx.Rollback()

    // 1. หัก Stock สินค้า (ตรวจสอบสต็อกเพียงพอก่อน)
    deductSQL := `UPDATE products SET stock = stock - $1 WHERE id = $2 AND stock
    >= $1`
    res, err := tx.Exec(deductSQL, req.Quantity, req.ProductID)
    if err != nil {
        return 0, err
    }

    affected, err := res.RowsAffected()
    if err != nil || affected == 0 {
        return 0, fmt.Errorf("product out of stock or product not found")
    }

    // 2. สร้างแถวรายการคำสั่งซื้อใหม่ (Order Record)
    var orderID int
    createOrderSQL := `INSERT INTO orders (product_id, quantity, status) VALUES
    ($1, $2, 'PAID') RETURNING id`
    err = tx.QueryRow(createOrderSQL, req.ProductID,
    req.Quantity).Scan(&orderID)
    if err != nil {
        return 0, err
    }

    // 3. ยืนยัน Transaction
    if err := tx.Commit(); err != nil {
        return 0, err
    }

    return orderID, nil
}

```

4. 🌸 การตั้งค่า Isolation Level ด้วย db.BeginTx()

หากต้องการกำหนดระดับความเป็นอิสระของการทำรายการ (Isolation Level) หรือใส่ `context.Context` ให้ใช้ `db.BeginTx()` :

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

// กำหนด Isolation Level เป็น Read Committed (หรือ RepeatableRead / Serializable)
opts := &sql.TxOptions{
    Isolation: sql.LevelReadCommitted,
    ReadOnly:  false,
}

tx, err := db.BeginTx(ctx, opts)
if err != nil {
    return err
}
defer tx.Rollback()
```

5. 🔒 ปัญหา Deadlock ใน Database Transaction และแนวทางแก้ไข

Deadlock คืออะไร?

Deadlock เกิดขึ้นเมื่อ Transaction ตั้งแต่ 2 ตัวขึ้นไปมีการแย่งชิง Lock ข้อมูลในลักษณะเป็นวงกลม (Circular Wait) โดยต่างฝ่ายต่างรอให้อีกฝ่ายคืน Lock ข้อมูลที่ตนเองต้องการ ทำให้ระบบติดค้าง (Stuck) ไม่สามารถทำงานต่อได้

ตัวอย่างสถานการณ์ Deadlock (Classic Scenario):

- **Transaction A:** ล็อกบัญชี 1 ➡ กำลังรอเพื่อล็อกบัญชี 2
- **Transaction B:** ล็อกบัญชี 2 ➡ กำลังรอเพื่อล็อกบัญชี 1
- **ผลลัพธ์ใน PostgreSQL:** จะเกิดข้อผิดพลาด Code `40P01` (`deadlock detected`) และระบบจะสั่ง Abort สุ่มยกเลิก Transaction ตัวใดตัวหนึ่งทันที

แนวทางป้องกันและการแก้ไข Deadlock (Deadlock Prevention Strategies)

1) การจัดเรียงลำดับการ Lock ให้เหมือนกันเสมอ (Consistent Lock Ordering)

บังคับให้ทุก Transaction อัปเดตหรือเข้าถึงข้อมูลตามลำดับ ID จากน้อยไปมากเสมอ ไม่ว่าจะโอนเงินจากใครไปหาใคร:

```

func TransferMoneySafe(db *sql.DB, fromID, toID int, amount float64) error {
    tx, err := db.Begin()
    if err != nil {
        return err
    }
    defer tx.Rollback()

    // จัดเรียง ID จากน้อยไปมากก่อนอัปเดตเสมอ เพื่อป้องกันการติด Deadlock แบบ Circular
    Wait
    firstID, secondID := fromID, toID
    if firstID > secondID {
        firstID, secondID = toID, fromID
    }

    // ล็อคและอัปเดตเรียงตาม ID
    if _, err := tx.Exec("UPDATE accounts SET balance = balance + $1 WHERE id =
$2", getAmount(firstID, fromID, amount), firstID); err != nil {
        return err
    }
    if _, err := tx.Exec("UPDATE accounts SET balance = balance + $1 WHERE id =
$2", getAmount(secondID, fromID, amount), secondID); err != nil {
        return err
    }

    return tx.Commit()
}

```

2) การทำระบบ Retry Mechanism (เมื่อเจอ Deadlock Error 40P01)

เมื่อเจอ Deadlock ฝั่ง Application ควรทำการสั่งรันลองใหม่ (Retry) โดยอัตโนมัติ:

```

func ExecTxWithRetry(db *sql.DB, maxRetries int, fn func(tx *sql.Tx) error)
error {
    for i := 0; i < maxRetries; i++ {
        tx, err := db.Begin()
        if err != nil {
            return err
        }

        err = fn(tx)
        if err == nil {
            return tx.Commit()
        }

        tx.Rollback()
        // หากเป็น Deadlock Error (40P01) ให้รอเวลาแล้ววนลูปทำงานใหม่ (Retry)
        if strings.Contains(err.Error(), "40P01") ||
strings.Contains(err.Error(), "deadlock") {
            time.Sleep(time.Duration(i+1) * 50 * time.Millisecond) //
Exponential Backoff
            continue
        }

        return err // Error อื่นๆ ที่ไม่ใช่ Deadlock ค็นค่าออกไปทันที
    }
    return fmt.Errorf("transaction failed after max retries")
}

```

6. ⚠️ ข้อควรระวังและแนวปฏิบัติที่ดี (Best Practices)

สำคัญ

1. **เรียกใช้เมธอดผ่านวัตถุ tx เสมอ:** ต้องใช้ `tx.Exec()` , `tx.Query()` , หรือ `tx.QueryRow()` แทน `db.Exec()` / `db.Query()` มิฉะนั้นคำสั่งนั้นจะอยู่นอก Transaction
2. **ใช้ `defer tx.Rollback()` เสมอ:** วางไว้บรรทัดถัดจาก `db.Begin()` ทันที เพื่อการันตีว่าหากเกิด error หรือ panic กลางคราว ระบบจะ Rollback คืนค่าข้อมูลให้อัตโนมัติ
3. **อย่าเปิด Transaction ค้างไว้นานเกินไป:** ไม่ควรใส่คำสั่งที่ใช้เวลานาน (เช่น การส่ง Email, การยิง HTTP Request ภายนอก) ไว้ในระหว่าง Transaction เพราะจะทำให้เกิดการ Lock แถวในตาราง DB นานเกินไป
4. **จัดลำดับการเรียง ID:** เมื่อทำ Transaction ที่มีการอัปเดตหลายแถว ควรเรียงลำดับ ID จากน้อยไปมาก เสมอเพื่อป้องกัน Deadlock

การใช้งาน Stored Procedure & Function ใน Go (Stored Procedure & Function in Go)

สรุปการสร้าง **Stored Procedure / Function** ใน PostgreSQL (PL/pgSQL) และวิธีการเรียกใช้งานจากภาษา Go ด้วยแพ็คเกจมาตรฐาน `database/sql`

1. ความแตกต่างระหว่าง Stored Procedure และ Function ใน PostgreSQL

คุณลักษณะ (Feature)	Function (CREATE FUNCTION)	Stored Procedure (CREATE PROCEDURE)
การเรียกใช้งานใน SQL	เรียกผ่าน <code>SELECT function_name(...)</code>	เรียกผ่าน <code>CALL procedure_name(...)</code>
การคืนค่า (Return Value)	ต้อง คืนค่าเสมอ (RETURNS scalar / TABLE / VOID)	ไม่บังคับให้คืนค่า (สามารถใช้ INOUT / OUT parameters ได้)
การควบคุม Transaction	ไม่สามารถใช้ COMMIT / ROLLBACK ภายในได้	สามารถใช้ COMMIT / ROLLBACK ภายในขั้นตอนได้ (PostgreSQL 11+)
การใช้ใน Query	สามารถนำไปต่อใน WHERE , JOIN หรือ SELECT ได้	ไม่สามารถนำไปซ้อนในคำสั่ง SELECT อื่นได้

2. การสร้าง Function & Stored Procedure ใน PostgreSQL

1) การสร้าง PL/pgSQL Function (คำนวณและคืนค่า)

```
-- Function คำนวณราคารวมสินค้าพร้อมภาษี (VAT 7%)
CREATE OR REPLACE FUNCTION calculate_total_with_vat(p_price NUMERIC, p_qty INT)
RETURNS NUMERIC AS $$
BEGIN
    RETURN (p_price * p_qty) * 1.07;
END;
$$ LANGUAGE plpgsql;

-- Function ดึงรายการสินค้าของ Supplier (คืนค่าเป็น TABLE)
CREATE OR REPLACE FUNCTION get_products_by_supplier(p_supplier_id INT)
RETURNS TABLE (product_id INT, product_name VARCHAR, price NUMERIC, stock INT)
AS $$
BEGIN
    RETURN QUERY
    SELECT p.id, p.name, p.price, p.stock
    FROM products p
    WHERE p.supplier_id = p_supplier_id
    ORDER BY p.id;
END;
$$ LANGUAGE plpgsql;
```

2) การสร้าง Stored Procedure (ย้าย Stock ระหว่าง Supplier)

```
CREATE OR REPLACE PROCEDURE transfer_product_supplier(
    p_product_id INT,
    p_new_supplier_id INT
)
AS $$
BEGIN
    UPDATE products
    SET supplier_id = p_new_supplier_id
    WHERE id = p_product_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Product ID % not found', p_product_id;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

3. การเรียกใช้งานจากภาษา Go (database/sql)

1) การเรียก Function ที่คืนค่าเดียว (Scalar Value)

ใช้คำสั่ง `SELECT function_name($1, $2)` ร่วมกับ `db.QueryRow().Scan()` :

```
func GetTotalWithVat(db *sql.DB, price float64, qty int) (float64, error) {
    var total float64
    query := `SELECT calculate_total_with_vat($1, $2)`
    err := db.QueryRow(query, price, qty).Scan(&total)
    if err != nil {
        return 0, fmt.Errorf("failed to call function: %w", err)
    }
    return total, nil
}
```

2) การเรียก Function ที่คืนค่าเป็นตาราง (Table Result)

ใช้คำสั่ง `SELECT * FROM function_name($1)` ร่วมกับ `db.Query()` และ `for rows.Next()` :

```

type ProductSimple struct {
    ID      int      `json:"id"`
    Name    string   `json:"name"`
    Price   float64  `json:"price"`
    Stock   int      `json:"stock"`
}

func GetProductsBySupplierFunc(db *sql.DB, supplierID int) ([]ProductSimple,
error) {
    query := `SELECT * FROM get_products_by_supplier($1)`
    rows, err := db.Query(query, supplierID)
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var products []ProductSimple
    for rows.Next() {
        var p ProductSimple
        if err := rows.Scan(&p.ID, &p.Name, &p.Price, &p.Stock); err != nil {
            return nil, err
        }
        products = append(products, p)
    }

    if err := rows.Err(); err != nil {
        return nil, err
    }

    return products, nil
}

```

3) การเรียก Stored Procedure (CALL)

ใช้คำสั่ง `CALL procedure_name($1, $2)` ร่วมกับ `db.Exec()` :

```

func TransferProductSupplier(db *sql.DB, productID, newSupplierID int) error {
    query := `CALL transfer_product_supplier($1, $2)`
    _, err := db.Exec(query, productID, newSupplierID)
    if err != nil {
        return fmt.Errorf("failed to call procedure: %w", err)
    }
    return nil
}

```

4. ⚖️ ข้อดีและข้อเสียของการใช้ Stored Procedure

ข้อดี (Pros)	ข้อเสีย (Cons)
Performance สูง: ประมวลผลบน Database Server โดยตรง ไม่ต้องส่งข้อมูลไปกลับหลายรอบข้าม Network	Vendor Lock-in: ย้ายไปใช้ Database Engine อื่นยาก (เช่น ย้ายจาก Postgres ไป MySQL ต้องเขียนใหม่)
Security: เพิ่มความปลอดภัยโดยการจำกัดสิทธิ์ให้ User เรียกใช้เฉพาะ Procedure แทนการเข้าถึง Table ตรงๆ	Version Control & Maintenance ยาก: จัดการและ Debug โค้ดใน DB ยากกว่าโค้ดฝั่ง Go
Reduce Business Logic Duplication: แชร์ logic ให้แอปหลายภาษา (Go, Node.js, Python) ใช้ร่วมกันได้	Testing ยาก: ไม่สามารถทำ Unit Test แบบ Mock ได้ง่ายเท่า Business Logic ฝั่ง Application

5. ⚠️ ข้อควรจำสำคัญ (Best Practices)

สำคัญ

1. การเรียกใช้ใน Go:

- **Function:** เรียกด้วย `SELECT function_name(...)` หรือ `SELECT * FROM function_name(...)`
- **Procedure:** เรียกด้วย `CALL procedure_name(...)` ร่วมกับ `db.Exec()`

2. Parameterized Query: ส่งพารามิเตอร์แบบ \$1, \$2 เสมอ เพื่อป้องกัน SQL Injection

3. Error Handling: หาก Procedure มีการโยน Error ออกมาด้วย RAISE EXCEPTION ใน PostgreSQL ฝั่ง Go จะได้รับ Error นั้นในตัวแปร err ทันที